

Dossier De Conception (DDC)

du projet

Robot Mini-Sumo

Responsabilité documentaire

Action	NOM Prénom	Fonction	Date	Signature
Rédigé par	Irastorza Iban Jobard Léo Blanc Arthur Clause Jules Feugnet Théo	Technicien	05/11/2025	
Approuvé par	Champion Frédéric Combastel Christophe	Chef de projet	05/11/2025	
Approuvé par	Champion Frédéric Combastel Christophe	Client	05/11/2025	

Suivi des révisions documentaires

Indice	Date	Nature de la révision
1	01/09/2022	Publication préliminaire du DDC document à compléter par le Technicien.
2	05/11/2025	Première publication

Documents de références

Sigle	Référence	Titre	Rév.	Origine
[CDC]	RMS_CDC	Cahier des charges	1	IUT GEii Bdx

Table des matières

Nature du document	5
Conception préliminaire du produit	5
2.1 Architecture Électronique	5
2.1.1 Choix des capteurs adversaire	7
2.1.2 Choix des capteurs de sol	8
2.1.3 Choix du pont en H	9
2.1.4 Choix du régulateur de tension	10
2.1.5 Choix de la batterie	11
2.2 Architecture Informatique	12
2.2.1 Fonctions d'acquisition	13
2.2.2 Fonctions d'action	13
2.2.3 Fonctions d'énergie	13
2.2.4 Fonctions de traitement	13
2.3 Conclusion de la conception préliminaire du produit	14
Conception détaillée du produit	15
3.1 Conception détaillée du système sonore et lumineux	15
3.2 Conception détaillée de la partie acquisition	16
3.2.1 Capteurs adversaires	16
3.2.1.1 Schéma électrique des capteurs adversaires	16
3.2.1.2 Code informatique des capteurs adversaires	16
3.2.2 Capteurs de sol (si nécessaire)	16
3.2.2.1 Schéma électrique des capteurs de sol	16
3.2.2.2 Code informatique des capteurs de sol	16
3.2.3 Capteurs de télécommande (reprendre le DDF de M. Blanchard)	16
3.2.3.1 Schéma électrique des capteurs de télécommande	16
3.2.3.2 Code informatique des capteurs adversaires	17
3.3 Conception détaillée de la partie action	18
3.3.1 Pont en H	18
3.3.1.1 Schéma électrique du pont en H	18
3.3.1.2 Code informatique du pont en H	19
3.4 Conception détaillée de la partie énergie	19
3.4.1 Pont diviseur de tension	19
3.4.1.1 Schéma électrique du pont diviseur de tension	19
3.4.1.2 Code informatique du pont diviseur de tension	19
3.4.2 régulateur de tension	19
3.4.2.1 Schéma électrique du régulateur de tension	19
3.5 Conception détaillée de la partie traitement	19

3.5.1 Le microcontrôleur ATMEGA328P-PU	19
3.4.1.1 Schéma électrique du microcontrôleur	20
3.3.1.2 Code informatique du microcontrôleur	20
Dérivage des solutions techniques retenues	21
4.1 <Titre de la simulation / prototypage rapide>	21
4.2 <Titre de la simulation / prototypage rapide>	22
4.3 Conclusion de la simulation / prototypage rapide du produit	24
Conclusion de la conception du produit	25
Matrice de conformité du produit	25

1. Nature du document

Ce document est un dossier de conception et a pour but de détailler la conception du produit développé. Il apporte ainsi des preuves de la conformité du produit par rapport à l'ensemble des exigences client. Le paragraphe 3 du [CDC] décrit de façon plus détaillée la nature et le positionnement de ce document dans l'arborescence documentaire du projet.

2. Conception préliminaire du produit

Ce chapitre décrit l'architecture fonctionnelle du produit. Il apporte les premiers éléments de preuve de la faisabilité du produit vis-à-vis des exigences client.

2.1 Architecture Électronique

Référence de pré-conception : CPR_C1a3, CPR_C1a9, CPR_C1a11

Exigences client vérifiées par préconception :

Rédacteurs : Arthur Blanc / Léo Jobard / Jules Clause / Théo Feugnet / Iban Irastorza

Relecteurs : Arthur Blanc / Léo Jobard / Jules Clause / Théo Feugnet / Iban Irastorza

Exigences d'acquisition d'information:

[EXIG_ADVERSAIRE](#), [EXIG_LUMINOSITE](#), [EXIG_DEPART](#)

Exigences de traitement d'informations:

[EXIG_COMPORTEMENT](#)

Exigences d'action:

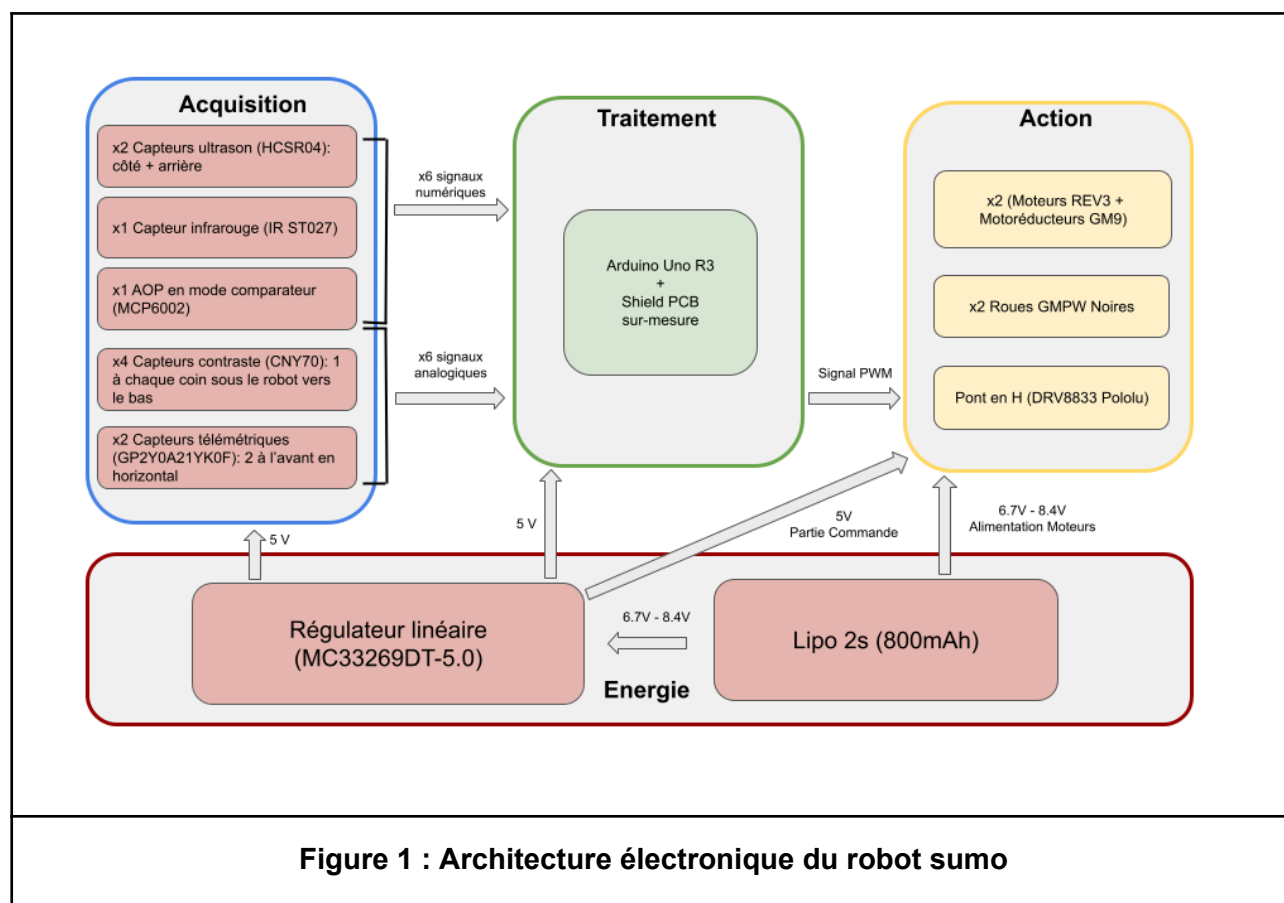
[EXIG_DEPLACEMENT](#)

Exigences énergétiques:

[EXIG_AUTONOMIE](#), [EXIG_SECUR_BATT](#)

Afin de répondre au cahier des charges, une analyse globale des exigences a conduit à l'architecture fonctionnelle présentée ci-dessous.

Robot Mini-Sumo (RMS)



Dans la partie acquisition, afin de répondre à l'EXIG_ADVERSAIRE, nous avons choisi d'intégrer deux capteurs télémétriques en analogique afin d'avoir un champ de détection large et précis. Nous avons notamment choisi d'intégrer à notre robot sumo deux capteurs ultrasons en numérique : un situé à l'arrière pour détecter l'adversaire à n'importe quel moment et l'autre situé sur le côté gauche afin de détecter l'adversaire au début de la manche. Nous avons choisi de placer quatre capteurs de contraste en analogique à chaque coin sous la base mécanique, orientés vers le bas afin de détecter la ligne blanche pour ne pas sortir du dohyo. Nous avons aussi ajouté un capteur infrarouge afin de répondre à l'EXIG_DEPART et ainsi recevoir le signal NEC émis par la télécommande qui nous sera donnée. Enfin, nous avons choisi d'utiliser un AOP en mode comparateur afin de répondre à l'EXIG_SECUR_BATT et ainsi détecter si le niveau de batterie est inférieur à 6,7 V.

Dans la partie traitement, nous utiliserons une carte Arduino Uno R3 avec un shield afin de répondre à l'EXIG_COMPORTEMENT et ainsi prendre des décisions en fonction des informations provenant de la partie acquisition.

Dans la partie action, deux moteurs avec chacun un motoréducteur et une roue nous sont imposés pour répondre à l'EXIG_DEPLACEMENT. Nous utiliserons un pont en H afin de contrôler les moteurs.

Dans la partie énergie, pour répondre à l'EXIG_AUTONOMIE, nous avons choisi d'utiliser un pack LiPo 2S 800 mAh pour alimenter les moteurs en 7,4 V et le reste du robot en 5 V à l'aide d'un régulateur linéaire.

2.1.1 Choix des capteurs adversaire

Référence de pré-conception : CPR_C1a4

Exigences client vérifiées par préconception : EXIG_ADVERSAIRE

Rédacteurs : Iban Irastorza / Jules Clause

Relecteurs : Arthur Blanc / Léo Jobard / Théo Feugnet

FICHE D'AIDE A LA DECISION (FAD)									
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)						Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Distance min et max de détection	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité		
Capteur de mesure Sharp GP2Y0A41SK0F	0	2-30cm	0.25 0.4 0.55 V	-0.3/+7V	12mA	9,90€	Oui	0,00	Ne répond pas à l'exigence de distance de détection
Capteur de mesure Sharp GP2Y0A021YK	1	10- 80 cm	0.25 0.4 0.55 V	-0.3/+7V	30mA	11,50€	Oui	5359,38	2 capteurs
Capteur de mesure Sharp GP2Y0A02YK	0	40- 150 cm	0.25 0.4 0.55 V	-0.3/+7V	33mA	12,80€	Oui	0,00	Détection trop lointaine
Capteur Ultrasonique HCSR04	1	2cm - 400cm	Input TTL lever signal and the range in proportion	5V	15mA	3,90€	Oui	10000,00	Trop peu précis, ne permet pas de déterminer si l'adversaire est parfaitement devant

Figure 2 : FAD pour le choix du capteur avant

Nous avons décidé d'utiliser deux capteurs télémétriques et de les configurer pour renvoyer une valeur TOR, car ils nous permettront de repérer le robot de manière plus précise. En effet, lorsque l'un des capteurs renverra la valeur 1, il nous suffira de faire

Robot Mini-Sumo (RMS)

tourner le robot dans un sens, puis de détecter l'adversaire avec les deux capteurs télémétriques avant d'avancer vers lui.

FICHE D'AIDE A LA DECISION (FAD)									
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)						Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Distance min et max de détection	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité		
Capteur de mesure Sharp GP2Y0A41SK0F	0	2-30cm	0.25 0.4 0.55 V	-0.3/+7V	12mA	9.90€	Oui	0,00	Ne répond pas à l'exigence de distance de détection
		0	3,5	5	4,5	4	5		
Capteur de mesure Sharp GP2Y0A021YK	1	10- 80 cm	0.25 0.4 0.55 V	-0.3/+7V	30mA	11.50€	Oui	5359,38	Trop cher
		5	3,5	5	3,5	3,5	5		
Capteur de mesure Sharp GP2Y0A02YK	0	40- 150 cm	0.25 0.4 0.55 V	-0.3/+7V	33mA	12.80€	Oui	0,00	Détection trop lointaine
		3	3,5	5	3	3	5		
Capteur Ultrasonique HCSR04	1	2cm - 400cm	Input TTL lever signal and the range in proportion	5V	15mA	3.90€	Oui	10000,00	1 capteur choisi
		5	4	5	4	5	5		

Figure 3 : FAD pour le choix du capteur adversaire arrière

Ce capteur nous aide à repérer le robot adverse plus rapidement lorsque l'on tourne et si l'adversaire est repéré par le capteur arrière alors le robot fera demi-tour.

FICHE D'AIDE A LA DECISION (FAD)									
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)						Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Distance min et max de détection	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité		
Capteur de mesure Sharp GP2Y0A41SK0F	0	2-30cm	0.25 0.4 0.55 V	-0.3/+7V	12mA	9.90€	Oui	0,00	Ne répond pas à l'exigence de distance de détection
		0	3,5	5	4,5	4	5		
Capteur de mesure Sharp GP2Y0A021YK	1	10- 80 cm	0.25 0.4 0.55 V	-0.3/+7V	30mA	11.50€	Oui	5359,38	Trop cher
		5	3,5	5	3,5	3,5	5		
Capteur de mesure Sharp GP2Y0A02YK	0	40- 150 cm	0.25 0.4 0.55 V	-0.3/+7V	33mA	12.80€	Oui	0,00	Détection trop lointaine
		3	3,5	5	3	3	5		
Capteur Ultrasonique HCSR04	1	2cm - 400cm	Input TTL lever signal and the range in proportion	5V	15mA	3.90€	Oui	10000,00	1 capteur choisi
		5	4	5	4	5	5		

Figure 4 : FAD pour le choix du capteur adversaire droite

Nous avons décidé de placer un capteur ultrason du côté droit du robot afin de détecter la présence de l'adversaire au tout début du combat. Si le capteur détecte l'adversaire, le robot reculera puis tournera vers la droite pour avancer rapidement vers lui. En revanche, si le capteur ne détecte rien, cela signifie que l'adversaire se trouve de l'autre côté, et le robot reculera alors avant de tourner vers la gauche.

2.1.2 Choix des capteurs de sol

Référence de pré-conception : CPR_C1a4

Exigences client vérifiées par préconception : EXIG_ADVERSAIRE, EXIG_LUMINOSITE

Rédacteurs : Iban Irastorza / Jules Clause

Relecteurs : Arthur Blanc / Léo Jobard / Théo Feugnet

Robot Mini-Sumo (RMS)

FICHE D'AIDE A LA DECISION (FAD)							
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)					Total (produit des valeurs précédentes)
		Distance min et max de détector	Compatibilité de la tension avec les sources disponibles	Respect de la stratégie de combat	Faible Prix	Disponibilité	
0 CNY avant + 0 CNY arrière		0mm - 10mm	oui	non	0	oui	0
		5	5	0	5	5	
1 CNY avant + 0 CNY arrière		0mm - 10mm	oui	non	0,71	oui	100
		5	5	1	4	5	
2 CNY avant + 0 CNY arrière		0mm - 10mm	oui	non	1,42	oui	150
		5	5	2	3	5	
0 CNY avant + 1 CNY arrière		0mm - 10mm	oui	non	0,71	oui	100
		5	5	1	4	5	
1 CNY avant + 1 CNY arrière		0mm - 10mm	oui	non	1,42	oui	75
		5	5	1	3	5	
2 CNY avant + 1 CNY arrière		0mm - 10mm	oui	non	2,13	oui	50
		5	5	1	2	5	
2 CNY avant + 2 CNY arrière		0mm - 575mm	oui	non	2,82	oui	200
		5	5	4	2	5	

Figure 5 : FAD pour le choix du capteur de ligne blanche du doyo

Nous avons choisi de placer 4 capteurs de contraste afin de ne pas dépasser les lignes. En effet, si une seule roue du robot sort du dohyo, la partie est terminée. Disposer de 4 capteurs de contraste nous permet de gérer la majorité des situations auxquelles le robot peut être confronté lorsqu'il se trouve au bord du terrain.

2.1.3 Choix du pont en H

Référence de pré-conception : CPR_C1a4

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

Rédacteurs : Théo Feugnet

Relecteurs : Arthur Blanc / Léo Jobard / Iban Irastorza / Jules Clause

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)					Total (produit des valeurs précédentes)
		Compatibilité de la tension avec les sources disponibles	Courant max de sortie du pont	Manière de piloter le pont	Faible Prix	Disponibilité	
Pololu Commande de 2 moteurs CC DRV8835 2x1,2A	1	Oui	1,5A	mode IN ou enable avec pwm	12,70€	oui	39,375
		1	2,5	4,5	3,5	1	
Pololu Commande de 2 moteurs CC DRV8833 2x1,2A	1	Oui	2A	mode IN avec PWM	11,40€	oui	54
		1	4	3	4,5	1	

Figure 6 : FAD pour le choix pont en H

D'après l'exigence EXIG_DEPLACEMENT, le robot doit être capable de se mouvoir dans n'importe quelle direction.

Pour cela nous devons choisir une solution technique capable de contrôler les moteurs de façon intelligente grâce au micro-contrôleur sans pour autant alimenter les moteurs avec ce dernier. Nous allons pour cela utiliser un pont H.

Notre choix de pont en H s'est porté sur le DV8833 plutôt que le DRV8835, pour les motifs suivants :

- Gestion des pics de courant : Les sollicitations intensives des moteurs, dues aux démarrages répétés et aux impacts avec l'adversaire, génèrent des pics de courant significatifs allant jusqu'à 160mA par moteur (mesurés lors de tests). De plus, même si nous n'allons probablement pas aller jusqu'à ces pics de courant, le constructeur indique un courant max du moteur GM3 avec motoréducteur RM3 est de 400mA. Les DV8833 (2A) et DV8835 (1.5A) ont tous les deux un courant maximal supérieur au courant en pic expérimental des deux moteurs associés (320mA) réduisant ainsi les risques de destruction des composants. co

- Pilotage en PWM : Les moteurs seront pilotés à l'aide des pins Timer/PWM du microcontrôleur arduino Uno. La commande par PWM rend donc superflue la fonction Enable du DRV8835. Le DV8833 répond alors pleinement aux exigences sans fonctionnalités inutiles.
- Contraintes d'alimentation : L'alimentation minimale est de 6,7 V d'après les exigences EXIG_AUTONOMIE et EXIG_SECUR_BATT. Les deux composants sont compatibles (DV8833 : 2,7-10,8 V et DRV8835 : -0,3-11 V), la différence de 0,2 V maximale étant négligeable car la tension maximale possible sortant de la batterie est de 8.4V.
- Aspect économique : Le DV8833 (11,40€) est inférieur de 1,30 € au DRV8835 (12,70€), consolidant ce choix sans impact sur les performances.

2.1.4 Choix du régulateur de tension

Référence de pré-conception : CPR_C1a4

Exigences client vérifiées par préconception : EXIG_AUTONOMIE

Rédacteurs : Théo Feugnet

Relecteurs : Arthur Blanc / Léo Jobard / Iban Irastorza / Jules Clause

Pour alimenter le système, une tension stable de 5V est nécessaire pour le bon fonctionnement des composants électroniques, à l'exception des moteurs qui seront directement alimentés par la batterie. Cette stabilité doit être assurée malgré la diminution progressive de la tension de la batterie au fil du match. Il doit aussi être capable de supporter le courant consommé par tous les autres composants.

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de (1) à (5))								Total it des valeurs précédentes
		Tension max d'entrée (V)	Tension de sortie (V)	Faible chute de tension sortie-entrée (V)	Courant de sortie max (A)	Bolier compact/léger à utiliser	Faible P _{dis} (W)	Disponibilité		
MC33269DT-3.3G IC	0	28	3.3	1	0.5	oui	0.7	oui		0
MC33269DT-5.0G IC	1	28	5	1	0.5	oui	N/A	oui (modéré)		6250

Figure 7 : FAD pour le choix du régulateur linéaire

Nous avons choisi le MC33269DT-5.0G IC car il régule la tension à 5V. De plus, il est capable de supporter suffisamment de courant en entrée :

$$I_{tot} = 2I_{\text{capteur télémétrique}} + 2I_{\text{capteur ultrason}} + 4I_{\text{capteur contraste}} + I_{\text{capteur IR}}$$

$$AN : I_{tot} = 2 \times 0.04 + 2 \times 0.015 + 4 \times 0.05 + 0.001 = 0.311 A = 311 mA$$

Il n'est actuellement plus en production mais nous possédons toujours des pièces en stock. Nous l'utiliserons donc tout de même.

Le MC33269DT-3.3G IC, bien que très similaire au MC33269DT-5.0G IC n'est pas conforme car il délivre une tension de sortie de 3.3V.

2.1.5 Choix de la batterie

Référence de pré-conception : CPR_C1a4

Rédacteurs : Iban Irastorza / Jules Clause / Théo Feugnet

Relecteurs : Arthur Blanc, Léo Jobard

Exigences client vérifiées par préconception : EXIG_AUTONOMIE

A	B	C	D	E	F	G
FICHE D'AIDE A LA DECISION (FAD)						
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)			Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Ensemblement	Faible Prix	Disponibilité		
Lipo CONRAD ENERGY 7.4 V 1200 mAh	1	31,104	17,99	oui	45	75g
		3	3	5		
Lipo CONRAD ENERGY 7.4 V 800 mAh	1	29,512	12,99	oui	80	52g
		4	4	5		
Lipo CONRAD ENERGY 7.4 V 450 mAh	0	19,954	8,29	oui	0	35g
		5	5	5		

Figure 8 : FAD pour le choix de la batterie

Nous avons établi la consommation totale de notre robot SUMO à partir des datasheets de ses composants:

$$I_{tot} = 2I_{\text{capteur télémétrique}} + 2I_{\text{capteur ultrason}} + 2 \times I_{\text{moteur}} + 4I_{\text{capteur contraste}} + I_{\text{capteur IR}}$$

Robot Mini-Sumo (RMS)

$$AN : I_{tot} = 2 \times 0.04 + 2 \times 0.015 + 2 \times 0.16 + 4 \times 0.05 + 0.001 = 0.631 A = 631 mA$$

$$Temps_{tot} = \frac{Capacité\ de\ la\ batterie}{I_{tot}} = \frac{0.8}{0.631} = 1.268h$$

$$Temps_{min} = 0.268 \times 60 = 16.1 minutes$$

$$Temps_{sec} = 0.1 \times 60 = 6 secondes$$

Donc :

$$Temps_{tot} = 1 h 16 min et 6 sec$$

Nous avons choisi de prendre la Lipo en 800mA.h car elle répond à l'exigence d'autonomie et elle est le meilleur compromis entre capacité et masse comme nous le montre la FAD ci-dessus.

2.2 Architecture Informatique

Référence de pré-conception : CPR_C1a3, CPR_C1a9, CPR_C1a11

Exigences client vérifiées par préconception : EXIG_SECUR_BATT, EXIG_ADVERSAIRE, EXIG_DEPART, EXIG_DEPLACEMENT, EXIG_COMPORTEMENT

Rédacteurs : Théo Feugne t/ Arthur Blanc / Léo Jobard / Jules Clause / Iban Irastorza

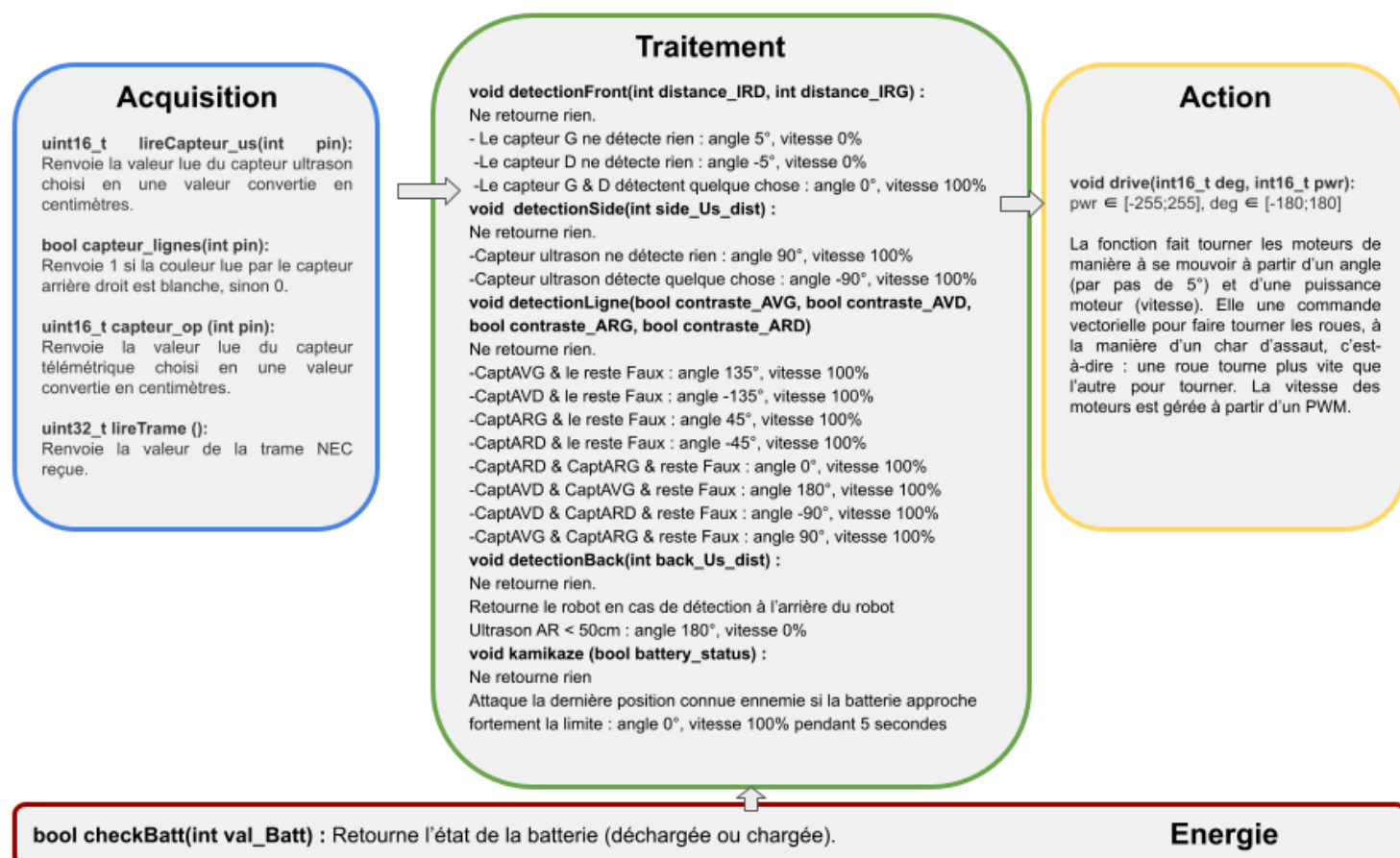


Figure 9 : Schéma de l'architecture informatique du robot sumo

Robot Mini-Sumo (RMS)

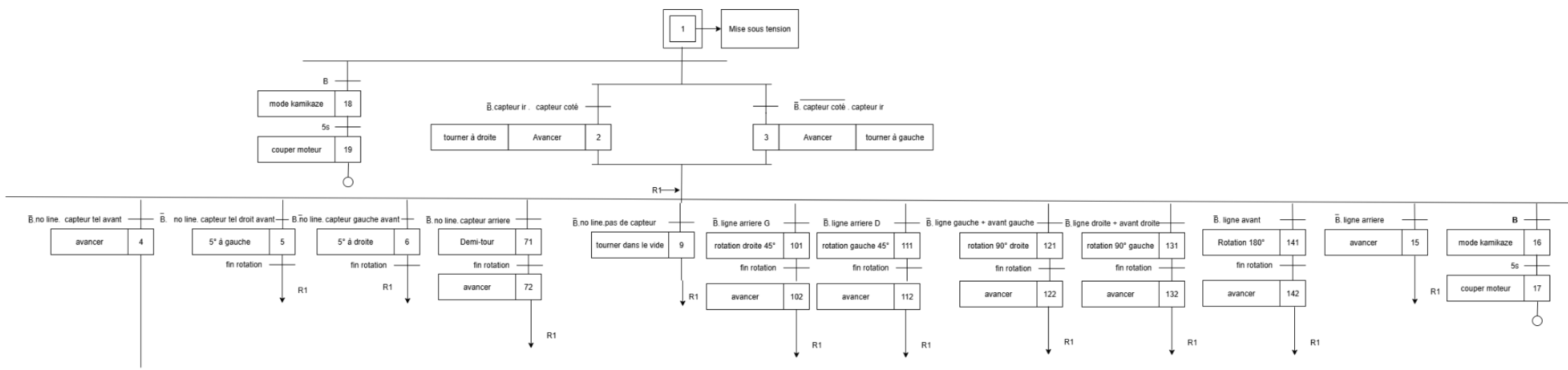


Figure 10 : grafcet de l'architecture informatique du robot sumo

2.2.1 Fonctions d'acquisition

Référence de pré-conception : CPR_C1a4, CPR_C1a10

Exigences client vérifiées par préconception : EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_DEPART

Rédacteurs : Iban Irastorza / Jules Clause / Théo Feugnet

Relecteurs : Arthur Blanc, Léo Jobard

uint16_t lireCapteur_us(int pin): renvoie la valeur lue du capteur ultrason choisi en une valeur convertie en centimètres

bool capteur_lignes(int pin): renvoie 1 si la couleur lue par le capteur arrière droit est blanche sinon 0

uint16_t capteur_op (int pin): renvoie la valeur lue du capteur télémétrique choisi en une valeur convertie en centimètres

uint32_t lireTrame (): renvoie la valeur de la trame NEC reçue

2.2.2 Fonctions d'action

Référence de pré-conception : CPR_C1a4, CPR_C1a10

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

Rédacteurs : Arthur Blanc, Léo Jobard

Relecteurs : Jules Clause, Théo Feugnet, Iban Irastorza

void drive(int deg, int pwr) :

La fonction fait tourner les moteurs de manière à se mouvoir à partir d'un angle (par pas de 5°) et d'une puissance moteur (vitesse). Elle utilise un différentiel de vitesse pour faire tourner les roues, à la manière d'un tank, c'est-à-dire : une roue tourne plus vite que l'autre pour tourner. La puissance moteur est gérée à partir d'un PWM.

2.2.3 Fonctions d'énergie

Référence de pré-conception : CPR_C1a4, CPR_C1a10

Exigences client vérifiées par préconception : EXIG_SECUR_BATT

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ22 Révision : 1 – 07/10/2025	17/55
----------------------------------	---	-------

Rédacteurs : Théo Feugnet

Relecteurs : Iban Irastorza / Jules Clause / Arthur Blanc / Léo Jobard

bool checkBatt(int val_Batt) :

retourne l'état de la batterie (déchargée ou chargée)

Vérifie le niveau de batterie par l'intermédiaire de la valeur d'un GPIO indiquant l'état de la tension de la batterie (0 : insuffisante car presque inférieure à 6.7V et 1 : si batterie suffisante car supérieure à 6.7V).

2.2.4 Fonctions de traitement

Référence de pré-conception : CPR_C1a4, CPR_C1a10

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT, EXIG_COMPORTEMENT

Rédacteurs : Théo Feugnet

Relecteurs : Iban Irastorza / Jules Clause / Arthur Blanc / Léo Jobard

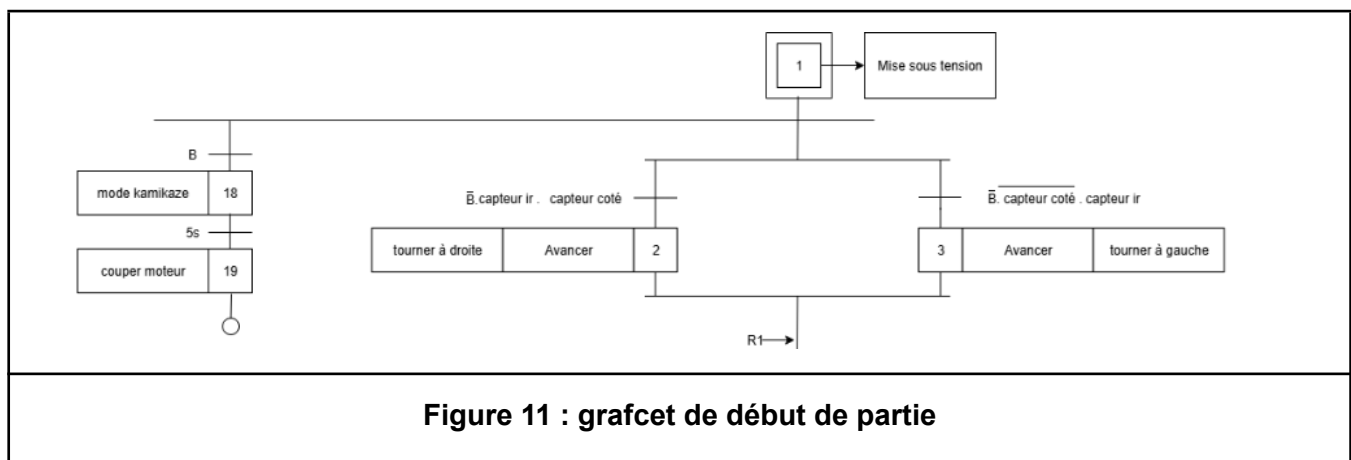
void detectionSide(int distance_UltrasonSide)

retourne rien

Traite les données du capteur ultrason du côté :

- Capteur ultrason ne détecte rien : angle 90° vitesse 100
Robot détecté à gauche, rotation de 90° dans le sens horaire puis moteurs à fond tout droit
- Capteur ultrason détecte quelque chose : angle -90° vitesse 100
Robot détecté à droite, rotation de 90° dans le sens trigonométrique puis moteurs à fond tout droit

Cette fonction n'est utilisée que lors du démarrage du robot afin de prédire la position du robot ennemi rapidement.

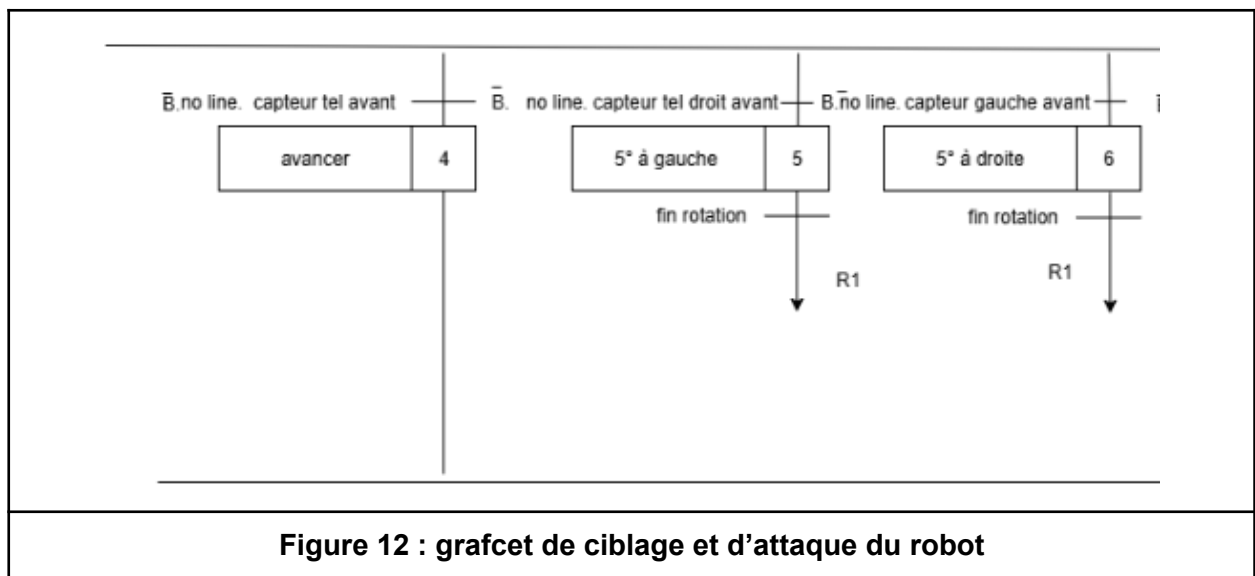


void detectionFront(int distance_Tel_Droit, int distance_Tel_Gauche)

retourne rien

Traite les données des capteurs télémétriques frontaux et corrige la trajectoire selon les valeurs des capteurs télémétriques :

- Le capteur gauche ne détecte rien : angle -5° vitesse 0
Cible repérée mais mal alignée : Correction par rotation horaire
- Le capteur droit ne détecte rien : angle 5° vitesse 0
Cible repérée mais mal alignée : Correction par rotation trigonométrique
- Le capteur droit et gauche détectent quelque chose : angle 0° vitesse 100
Cible repérée et correctement alignée : moteurs à fond tout droit
- Les capteurs droit et gauche ne détectent rien : angle 5° vitesse 0
Aucune cible repérée : rentrée en mode recherche, rotation sur lui même dans le sens trigonométrique



void detectionLigne(bool etat_CaptFG_Ligne, bool etat_CaptFD_Ligne, bool etat_CaptBG_Ligne, bool etat_CaptBD_Ligne)

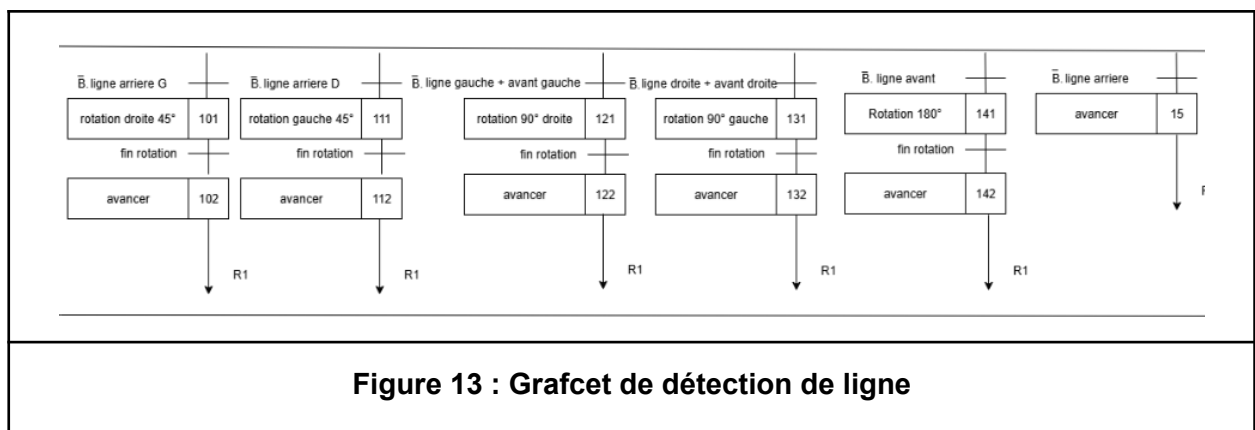
retourne rien

Traite les données et corrige la trajectoire selon les valeurs des capteurs de contraste :

- CaptFG & all Capt faux : angle -135° , vitesse 100%
Ligne détectée au coin avant gauche, rotation de -135° dans le sens horaire puis moteurs à fond tout droit

Robot Mini-Sumo (RMS)

- CaptFD & all Capt faux : angle 135° , vitesse 100%
Ligne détectée au coin avant droit, rotation de 135° dans le sens trigonométrique puis moteurs à fond tout droit
- CaptBG & all Capt faux : angle -45° , vitesse 100%
Ligne détectée au coin arrière droit, rotation de -45° dans le sens horaire puis moteurs à fond tout droit
- CaptBD & all Capt faux : angle 45° , vitesse 100%
Ligne détectée au coin arrière droit, rotation de 45° dans le sens trigonométrique puis moteurs à fond tout droit
- CaptBD & CaptBG & reste Faux : angle 0° vitesse 100%
Ligne détectée à l'arrière, aucune rotation puis moteurs à fond tout droit
- CaptFD & CaptFG & reste Faux : angle -180° vitesse 100%
Ligne détectée à l'avant, rotation de -180° dans le sens horaire puis moteurs à fond tout droit
- CaptFD & CaptBD & reste Faux : angle 90° vitesse 100%
Ligne détectée à droite, rotation de 90° dans le sens trigonométrique puis moteurs à fond tout droit
- CaptFG & Capt BG & reste Faux : angle -90° vitesse 100%
Ligne détectée à gauche, rotation de -90° dans le sens horaire puis moteurs à fond tout droit



void detectionBack(int distance_UltrasonBack)

retourne rien

Fait faire un demi-tour au robot en cas de détection à l'arrière du robot :

- Ultrason Back < distance cm (à définir) : angle 180° , vitesse 0
Cible détectée à l'arrière : retourner le robot et le viser.

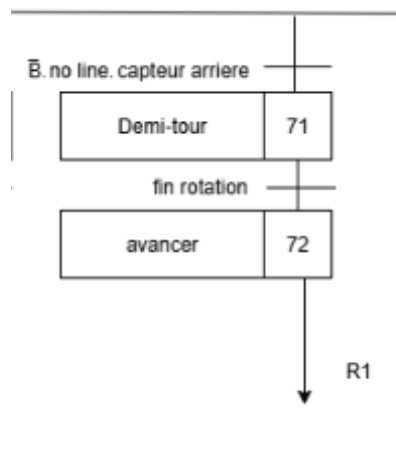


Figure 14 : Grafset de détection d'adversaire dans le dos

void kamikaze (bool battery_status)

retourne rien

Attaque la dernière position connue ennemie si la batterie approche fortement la limite :

- Vitesse 100% Angle 0° pendant 5 secondes

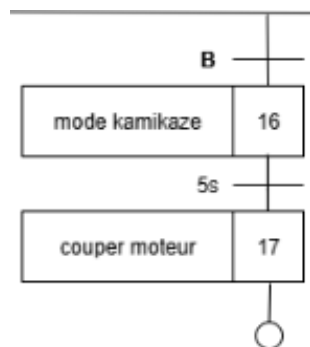


Figure 15 : Grafset de Kamikaze

2.3 Conclusion de la conception préliminaire du produit

La phase de conception préliminaire a permis de définir une architecture électronique et informatique conforme aux exigences du cahier des charges. Les choix techniques effectués pour les capteurs, le

Robot Mini-Sumo (RMS)

pont en H, le régulateur de tension et la batterie assurent le bon fonctionnement du robot tout en garantissant un équilibre entre performance et autonomie.

L'étude des différentes fonctions d'acquisition, de traitement, d'action et d'énergie a confirmé la faisabilité du système proposé. La majorité des exigences client (EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_DEPART, EXIG_DEPLACEMENT, EXIG_COMPORTEMENT, EXIG_AUTONOMIE, EXIG_SECUR_BATT) sont satisfaites à ce stade du projet.

Quelques ajustements pourront être envisagés lors de la conception détaillée afin d'optimiser la détection et la gestion énergétique. Dans l'ensemble, la conception préliminaire valide la cohérence et la viabilité technique du robot Mini-Sumo.

3. Conception détaillée du produit

Ce chapitre détaille la conception du produit développé. Il constitue une preuve de la conformité du produit. Chaque paragraphe de cette étude fait donc clairement référence aux exigences client issues du [CDC].

3.1 Conception détaillée du robot mini-sumo

Chaque bloc fonctionnel doit faire l'objet d'un chapitre de conception détaillé, présenté comme suit.

Référence de conception : CDT_C1B21, CDT_C1B22, CDT_C1B23,

A la suite de la conception préliminaire, l'activité de conception détaillée a été menée. Le schéma électrique complet de la carte est fourni ci-dessous.

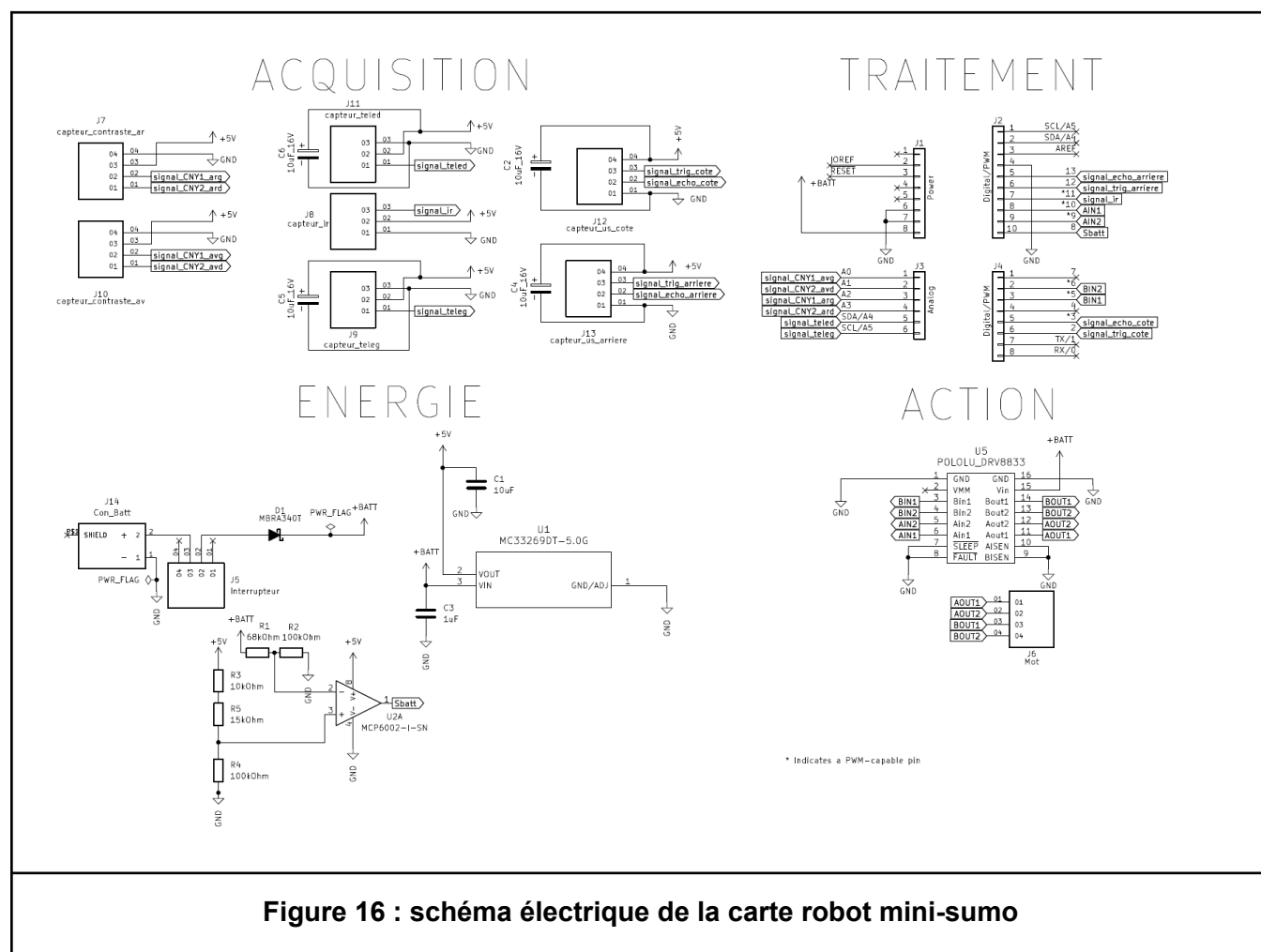


Figure 16 : schéma électrique de la carte robot mini-sumo

3.2 Conception détaillée de la partie acquisition

3.2.1 Capteurs adversaires

Référence de conception : CDT_C1B21, CDT_C1B22, CDT_C1B23,

Rédacteurs : Iban Irastorza / Jules Clause

Relecteurs : Arthur Blanc / Léo Jobard / Théo Feugnet

Exigences client vérifiées : [EXIG_ADVERSAIRE](#)

3.2.1.1 Schéma électrique des capteurs adversaires

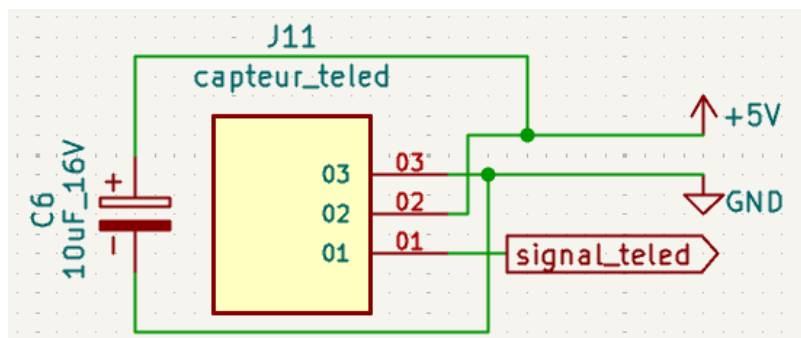


Figure 17 : Schéma électrique capteur télémétrique droit

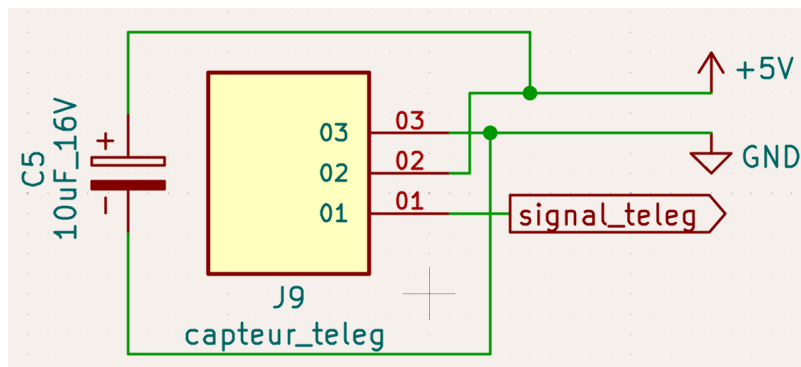


Figure 18 : Schéma électrique capteur télémétrique gauche

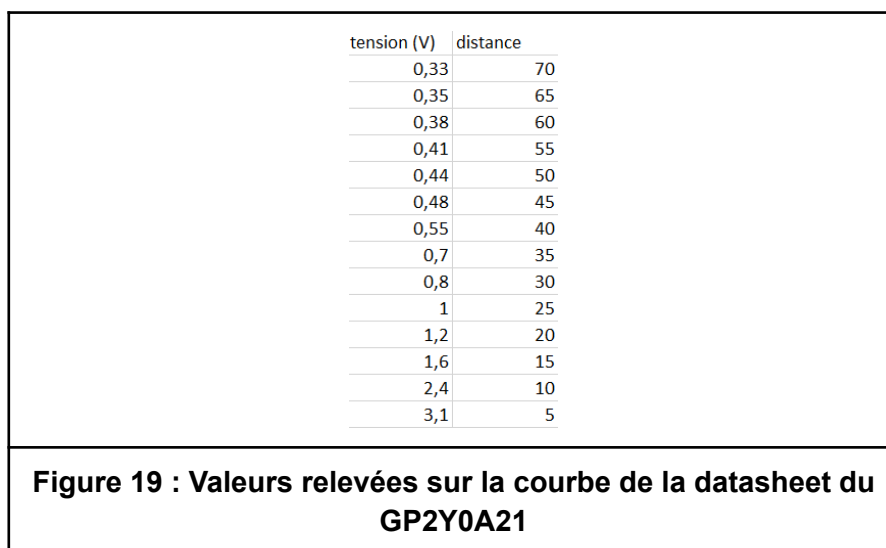
Pour répondre à l'exigence [EXIG_ADVERSAIRE](#), nous avons choisi d'intégrer deux capteurs télémétriques GP2Y0A21YK0F à l'avant de notre robot. Ces capteurs, dont la plage de mesure s'étend de 10 cm à 80 cm, permettent de détecter efficacement un adversaire à une distance supérieure à 40 cm.

Le capteur GP2Y0A21YK0F fonctionne selon le principe de triangulation optique : une LED infrarouge émet un faisceau vers un obstacle, et la lumière réfléchie est reçue par un photodiode placée sous un angle fixe. La position du point lumineux sur le capteur interne dépend de la distance de l'objet, ce qui permet au circuit intégré du module de convertir cette information en une tension analogique

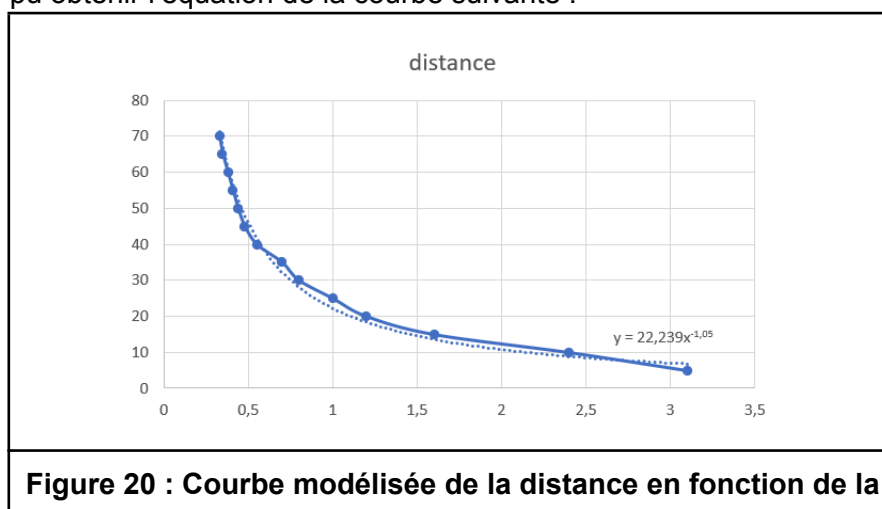
proportionnelle à la distance.

Sur le plan électronique, nous avons prévu sur KiCad des connecteurs à trois broches permettant de relier respectivement les capteurs GP2Y0A21YK0F droit et gauche au microcontrôleur. D'après la datasheet, l'alimentation est en 5V donc nous n'avons pas besoin de modifier la tension d'alimentation du composant, elle est située sur la broche centrale, la masse est sur la broche supérieure, et la sortie analogique (signal_telec et signal_teled) sur la broche inférieure. La datasheet recommande d'ajouter un condensateur de découplage de 10 μF (nous choisirons une valeur normalisée à 10 μF), placé au plus près du capteur entre ses bornes d'alimentation (Vcc et GND). Ce condensateur permet de stabiliser la tension d'alimentation, d'absorber les pics de courant liés à la LED infrarouge interne, et de réduire le bruit ou les oscillations sur la sortie analogique.

Pour aborder la partie informatique des capteurs GP2Y0A21YK0F, nous avons modélisé la courbe présente sur la [documentation](#) du GP2Y0A21YK0F de la tension de sortie en fonction de la distance d'un objet. Cela nous a permis d'obtenir une fonction qui normalise la valeur de la tension de sortie en centimètre. Nous avons mesuré les valeurs suivantes :



Enfin, nous avons pu obtenir l'équation de la courbe suivante :



tension du GP2Y0A21

Après avoir remarqué lors de tests que la précision était faible, nous avons expérimentalement ajusté le facteur 22,239 à 12 et la puissance à -1 pour obtenir des résultats cohérents sur notre plage de mesures.

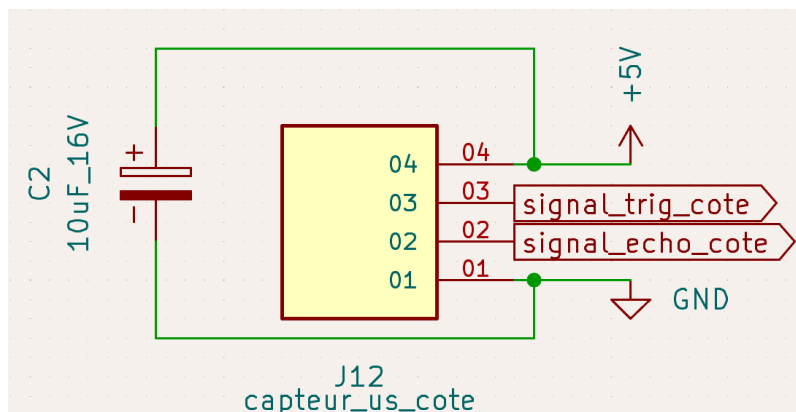


Figure 21 : Schéma électrique capteur ultrason côté

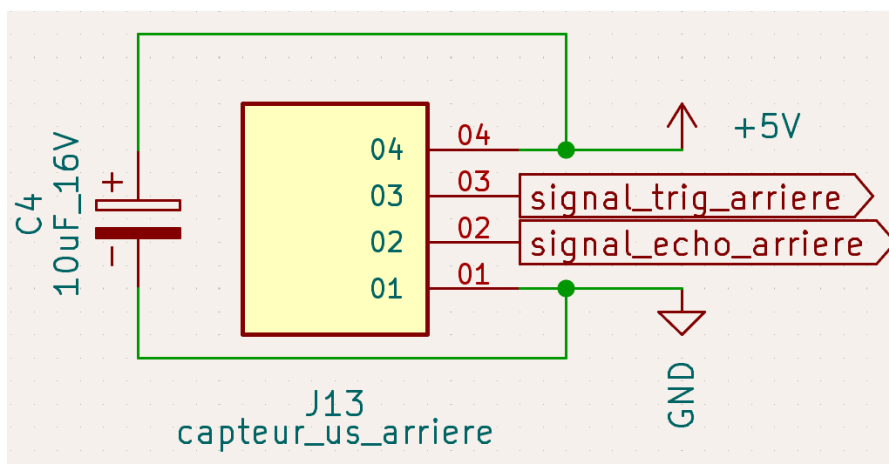


Figure 22 : Schéma électrique capteur ultrason arrière

Pour répondre à l'exigence [EXIG AVERSAIRE](#), nous avons intégré deux capteurs ultrasons HC-SR04, positionnés respectivement à l'arrière et sur le côté de notre robot. Ces capteurs permettent de mesurer la distance entre le robot et un obstacle grâce à l'émission et à la réception d'ondes ultrasonores. Leur plage de mesure s'étend de 2 cm à 400 cm, ce qui les rend adaptés à la détection d'obstacles proches.

Le fonctionnement du HC-SR04 repose sur la mesure du temps de vol (time of flight) d'une onde ultrasonore. Lorsqu'une impulsion est appliquée sur la broche Trig, le capteur émet une courte salve d'ultrasons à 40 kHz. L'écho réfléchi par un obstacle est ensuite capté par le récepteur, et la durée

entre l'émission et la réception est proportionnelle à la distance de l'objet détecté. Le microcontrôleur calcule alors la distance en utilisant la vitesse du son dans l'air.

Sur le plan électronique, nous avons prévu sur KiCad des connecteurs à quatre broches permettant de relier les capteurs HC-SR04 arrière et latéral au microcontrôleur. D'après la datasheet, le brochage est le suivant : le Vcc est en 5 V et en haut donc nous n'avons pas besoin de modifier la source d'alimentation du composant, les pin TRIGGER puis ECHO sont en dessous, et enfin la masse (GND) à l'extrémité basse. En nous servant du [HTUT 24](#), nous avons dimensionné son condensateur de découplage avec la formule suivante :

$$I = C * dU / dt \quad \text{donc} \quad C = I * dt / dU \text{ qui peut aussi s'écrire } C = I * (1/f) / \Delta V_{cc}$$

Avec f la fréquence de fonctionnement du composant et ΔV_{cc} étant inconnu, on prendra $\Delta V_{cc} = 0,01 * V_{cc}$. Sur la [datasheet](#) du HCSR04, on relève un courant de fonctionnement de 15 mA et une fréquence de fonctionnement de 40 kHz. On a donc :

$$C = (15 * 10^{-3} * 1 / 40000) / 0,01 * 5 = 7,5 \mu F, \text{ on prendra une valeur normalisée de } 10 \mu F.$$

3.2.1.2 Code informatique des capteurs adversaires

Télémétrie :

```
#include <math.h>

#define SEUIL_DISTANCE_A 50          // Seuil de détection en cm
const float referenceVoltage = 5.0;  // Tension de référence Arduino
const int pinCapteurGauche = A5;
const int pinCapteurDroit = A4;

void setup() {
    // Configuration des pins des capteurs
    pinMode(pinCapteurGauche, INPUT);
    pinMode(pinCapteurDroit, INPUT);
}

// Fonction de détection générique
bool detecterRobotIR(int pinCapteur) {

    float distance_moy = 0;
    for (int i = 0; i < 3; i++) {
        int valeurBrute = analogRead(pinCapteur);
        float tension = (valeurBrute / 1023.0) * referenceVoltage;
        float distance = 12 * pow(tension, -1);
        distance_moy += distance;
    }
    distance_moy = round(distance_moy / 3);

    if (distance_moy <= SEUIL_DISTANCE_A) {
        return 1; // Robot détecté
    } else {
        return 0; // Aucun robot
    }
}
```

```

}

void loop() {
  bool robotGauche = detecterRobotIR(pinCapteurGauche);
  bool robotDroit  = detecterRobotIR(pinCapteurDroit);
}

```

Figure 23 : code capteur télémétrique

La valeur analogique lue par `analogRead()` (comprise entre 0 et 1023) est convertie en tension, puis transformée en distance à l'aide de notre équation établie plus haut : `12 * pow(tension, -1)`. Cette relation permet d'obtenir une estimation fiable de la distance en centimètres à partir de la tension mesurée. Pour limiter les fluctuations dues au bruit, la fonction effectue trois lectures successives et en calcule la moyenne. Si la distance moyenne est inférieure à 50 cm, la fonction `detecterRobotIR()` renvoie 1, indiquant qu'un robot adverse est détecté dans le champ du capteur. Sinon, elle renvoie 0. Dans le `loop`, les deux variables `robotGauche` et `robotDroit` récupèrent les valeurs booléennes de la fonction pour chaque capteur. Cela reflètent donc en temps réel la présence d'un adversaire sur chaque côté avant du robot. Ces signaux binaires peuvent ensuite être directement exploités par le programme principal pour déclencher une action.

Ultrason :

```

#define TRIG_PINar 12
#define ECHO_PINar 13
#define TRIG_PINco 2
#define ECHO_PINco 3
#define SEUIL_DISTANCE_B 70 // Distance limite pour détecter un robot adverse (en cm)

void setup() {
  pinMode(TRIG_PINar, OUTPUT);
  pinMode(ECHO_PINar, INPUT);
  pinMode(TRIG_PINco, OUTPUT);
  pinMode(ECHO_PINco, INPUT);
}

// --- Définition de la fonction ---

int detecterRobot(int trigPin, int echoPin) {
  long duree;
  int distance;

  // Initialisation du signal TRIG
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Envoi d'une impulsion de 10 µs pour déclencher l'émission ultrasonore
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
}

```

```

digitalWrite(trigPin, LOW);

// Mesure du temps de retour de l'écho
duree = pulseIn(echoPin, HIGH);

// Conversion du temps mesuré en distance (cm)
distance = (duree * 0.0343) / 2;

// Retourne 1 si un robot est détecté dans la zone (moins de 70 cm), sinon 0
if (distance < SEUIL_DISTANCE_B) {
    return 1;
} else {
    return 0;
}
}

void loop() {
    int robotAvant = detecterRobot(TRIG_PINco, ECHO_PINco);
    int robotArriere = detecterRobot(TRIG_PINar, ECHO_PINar);
}

```

Figure 24 : code capteur ultrason

Chaque capteur émet une impulsion de 10 μs sur sa broche TRIG pour générer une onde sonore. Le temps de retour de cette onde, mesuré sur la broche ECHO par la fonction `pulseIn()`, correspond à l'aller-retour du signal dans l'air. Ce temps est converti en distance en multipliant par la vitesse du son (0,0343 cm/ μs) et en divisant par deux. Si la distance obtenue est inférieure à 70 cm, la fonction `detecterRobot()` renvoie 1, indiquant la détection d'un robot adverse ; sinon, elle renvoie 0. Dans le `loop`, les variables `robotAvant` et `robotArriere` récupèrent la valeur booléenne de la fonction pour chaque capteur. Ces valeurs logiques peuvent ensuite être exploitées pour déclencher des actions de combat. Le fonctionnement est continu : à chaque cycle de la boucle, les deux capteurs réalisent une mesure et mettent à jour l'état de détection en temps réel, assurant une réactivité maximale du robot pendant le combat.

3.2.2 Capteurs de sol

Référence de conception : CDT_C1B21, CDT_C1B22, CDT_C1B23,

Rédacteurs : Iban Irastorza / Jules Clause

Relecteurs : Arthur Blanc / Léo Jobard / Théo Feugnet

Exigences client vérifiées : [EXIG LUMINOSITE](#)

3.2.2.1 Schéma électrique des capteurs de sol

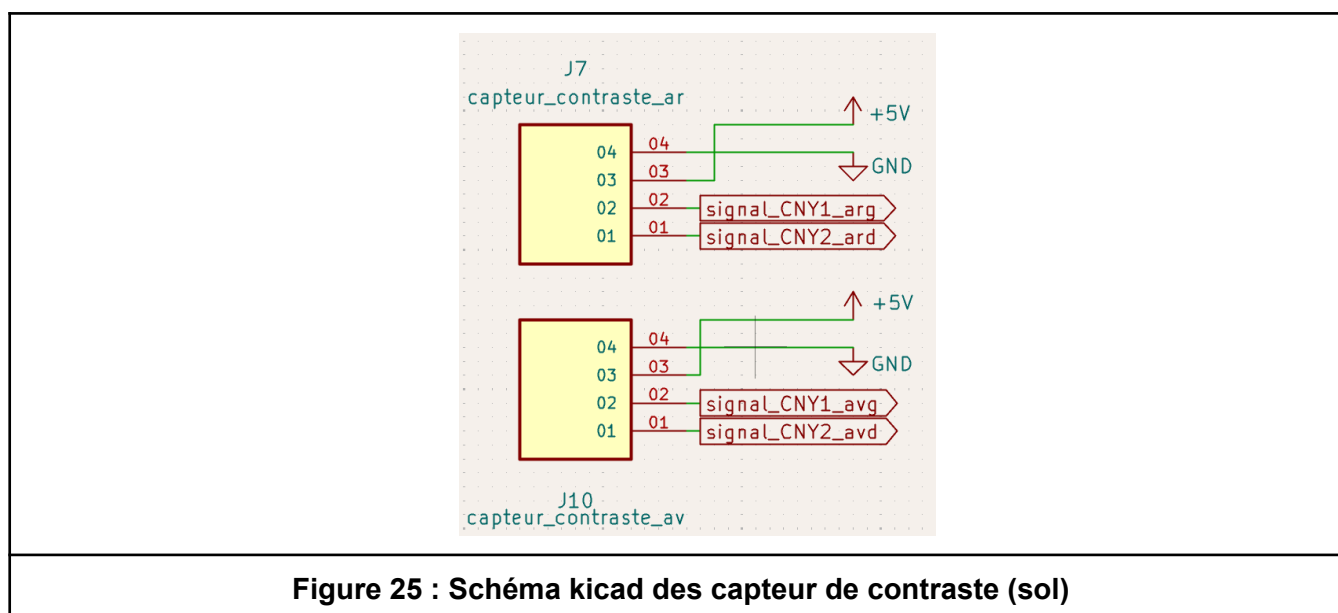


Figure 25 : Schéma kicad des capteur de contraste (sol)

Les capteurs ont été préalablement montés sur des cartes électroniques, où il était indiqué le type de sortie (Vcc, GND, CNY1, etc...) juste en face du pin. Sur la carte électronique, chaque capteur est doté de 2 résistances, une pour la LED de 180Ω et une pour le phototransistor de 4,7kΩ (cf. [DDF CNY70](#)). La LED infrarouge du CNY70 doit être protégée pour limiter son courant. Sans résistance, la LED consommerait trop et serait endommagée. D'après la [datasheet](#) du CNY70, les caractéristiques de la LED infrarouge sont:

- Quand $I_F = 20 \text{ mA}$, $V_F \approx 1,15 \text{ V}$
- $V_{CC} = 5 \text{ V}$

D'après la loi d'ohm : $R = (V_{CC} - V_F) / I_F = (5 - 1,15) / 0,02 = 192,5 \Omega$

La valeur est normalisée à 180 Ω.

On voit dans la datasheet que le capteur CNY70 fonctionne grâce à un système de réflexion infrarouge. La LED infrarouge éclaire la surface située en face, et le phototransistor mesure la lumière réfléchiée. Quand la surface est claire (blanche), elle réfléchit beaucoup de lumière : le phototransistor reçoit plus de rayonnement et laisse passer davantage de courant donc la tension de sortie augmente. Au contraire, quand la surface est foncée (noire), elle absorbe la lumière infrarouge : le phototransistor reçoit peu de lumière donc la tension de sortie diminue. La résistance de 4,7 kΩ permet donc d'obtenir une variation de tension suffisante entre surface blanche et surface noire et d'éviter que le transistor ne soit saturé trop vite. La masse du CNY70 est située sur l'extrémité haute, son alimentation (5V) est sur la broche du dessous, et les deux broches des signaux analogiques sont sur les deux broches du dessous. Après avoir effectué des tests, nous en avons déduit que le seuil de valeur analogique qui satisfait le mieux l'exigence est 800.

3.2.2.2 Code informatique des capteurs de sol

```
#define SEUIL 800
```

Robot Mini-Sumo (RMS)

```
const int CNY_avant_gauche  = A0;
const int CNY_avant_droit   = A1;
const int CNY_arriere_gauche = A2;
const int CNY_arriere_droit = A3;

void setup() {
    // Déclaration explicite des pins comme entrées
    pinMode(CNY_avant_gauche, INPUT);
    pinMode(CNY_avant_droit, INPUT);
    pinMode(CNY_arriere_gauche, INPUT);
    pinMode(CNY_arriere_droit, INPUT);
}

// Fonction de détection de couleur : retourne 1 si blanc, 0 si noir
bool detecterBlanc(int pinCapteur) {
    int valeur = analogRead(pinCapteur);
    return (valeur < SEUIL) ? 1 : 0;
}

void loop() {
    bool avantGauche  = detecterBlanc(CNY_avant_gauche);
    bool avantDroit   = detecterBlanc(CNY_avant_droit);
    bool arriereGauche = detecterBlanc(CNY_arriere_gauche);
    bool arriereDroit  = detecterBlanc(CNY_arriere_droit);
}
```

Figure 26 : Code informatique capteur de contraste

La fonction `detecterBlanc(int pinCapteur)` permet de transformer la lecture analogique d'un capteur CNY en un signal binaire indiquant la couleur de la surface détectée. Elle commence par lire la valeur brute du capteur à l'aide de `analogRead(pinCapteur)`, qui renvoie un entier compris entre 0 et 1023 proportionnel à la tension fournie. Ensuite, la valeur mesurée est comparée à un seuil prédéfini (`SEUIL = 800`). Si la valeur est inférieure au seuil, la fonction considère que la surface est blanche et renvoie 1. Si la valeur est supérieure ou égale au seuil, elle considère que la surface est noire et renvoie 0. Dans la boucle, les quatre variables binaires `avantGauche`, `avantDroit`, `arriereGauche` et `arriereDroit` sont mises à jour en continu et peuvent être directement exploitées pour déclencher des actions comme éviter de sortir du ring ou corriger la trajectoire.

3.2.3 Capteurs de télécommande

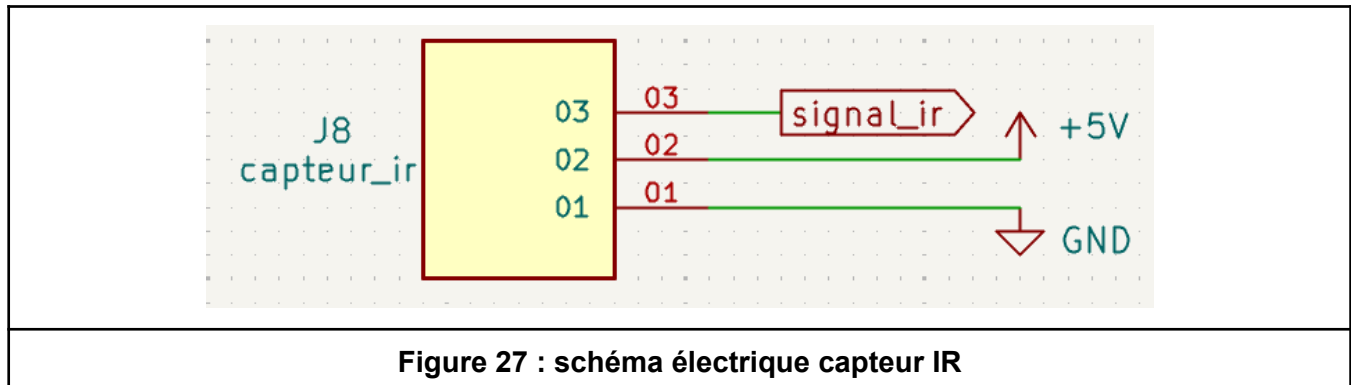
Référence de conception : CDT_C1B21, CDT_C1B22, CDT_C1B23,

Rédacteurs : Iban Irastorza / Jules Clause

Relecteurs : Arthur Blanc / Léo Jobard / Théo Feugnet

Exigences client vérifiées : [EXIG_DEPART](#)

3.2.3.1 Schéma électrique des capteurs de télécommande



Nous avons choisi d'utiliser le capteur Infrarouge IR ST027 pour répondre à l'[EXIG_DEPART](#). En effet, ce module didactique basé sur un récepteur IR permet la réception de signaux de télécommandes IR comme la [IRC01](#). Son alimentation 5V est située sur la broche du milieu, sa masse GND est à l'extrémité basse et sa sortie numérique "signal_ir" est à l'extrémité haute.

3.2.3.2 Code informatique des capteurs de télécommande

```
#include <IRremote.h>

// Variables
int Etat_LED = 0;
int receiverPin = 11;
decode_results results;
unsigned long lastTime = 0;

IRrecv irrecv(receiverPin);

void initIR() {
    irrecv.enableIRIn(); // Start the receiver
}

int lireIR() {
    if (irrecv.decode(&results)) {

        // Anti répétition (200ms)
        if (millis() - lastTime > 200) {
```



```

        lastTime = millis();
        irrecv.resume();
        return 1;
    }

    irrecv.resume();
}
return 0;
}

void setup() {
    initIR();
    pinMode(12, OUTPUT); // LED sur pin 12
    Serial.begin(9600);
}

void loop() {
    int info = lireIR();

    if (info == 1) {
        Serial.println("1");
    }
    Serial.println("0");
    delay(200);
}

```

Figure 28 : Code informatique LED infra-rouge

Dans ce code d'essai des capteurs infrarouges, nous faisons appel à une fonction setup, loop, et à deux fonctions qui sont : void initIR() et int lireIR(). Nous avons utilisé la bibliothèque IRremote.h. Cette dernière permet d'utiliser différentes fonctions afin de recevoir une donnée de notre télécommande.

Voici les différentes fonctions utilisées :

- initIR() : cette fonction fait appel à une fonction de la bibliothèque IRremote.h qui est irrecv.enableIRIn(). Cette dernière permet d'activer le capteur infrarouge afin d'être prêt à recevoir une donnée de la télécommande.
- Notre fonction lireIR() contient une autre fonction de la bibliothèque IRremote.h : IrReceiver.decode(). Celle-ci a pour but de décrypter les informations envoyées par notre télécommande.
- irrecv.resume() : cette fonction issue également de la bibliothèque IRremote.h permet d'actualiser les valeurs stockées afin d'être prêt à la réception d'une nouvelle donnée.

Le moniteur série renvoie 0 lorsque la variable "info" est à 0

Dans notre fonction, nous utilisons la fonction millis() afin d'être le plus réactif à l'appui sur la télécommande. En effet, la condition millis()-temps>200 (ici le temps correspond au temps de l'appui

précédent, nous remettons à jour le temps avec la valeur `millis()`, nous permet de faire appel à la fonction `irrecv.resume()` dès l'appui. Nous n'avons pas besoin d'attendre le relâchement du bouton. A chaque appui, l'état logique de la variable "info" passe à 1.

Pour conclure, ce code permet de vérifier si le récepteur infrarouge et à quel moment il exécute nos lignes de code avec l'allumage de notre LED. La LED remplace l'état des moteurs pour les besoins du test et représente si les moteurs sont en fonctionnement ou en arrêt.

3.3 Conception détaillée de la partie action

3.3.1 Pont en H

Référence de conception : CDT_PONTH

Exigences client vérifiées : EXIG_DEPLACEMENT

Rédacteurs : Arthur Blanc / Léo Jobard

Relecteurs : Iban Irastorza / Jules Clause / Théo Feugnet

3.3.1.1 Schéma électrique du pont en H

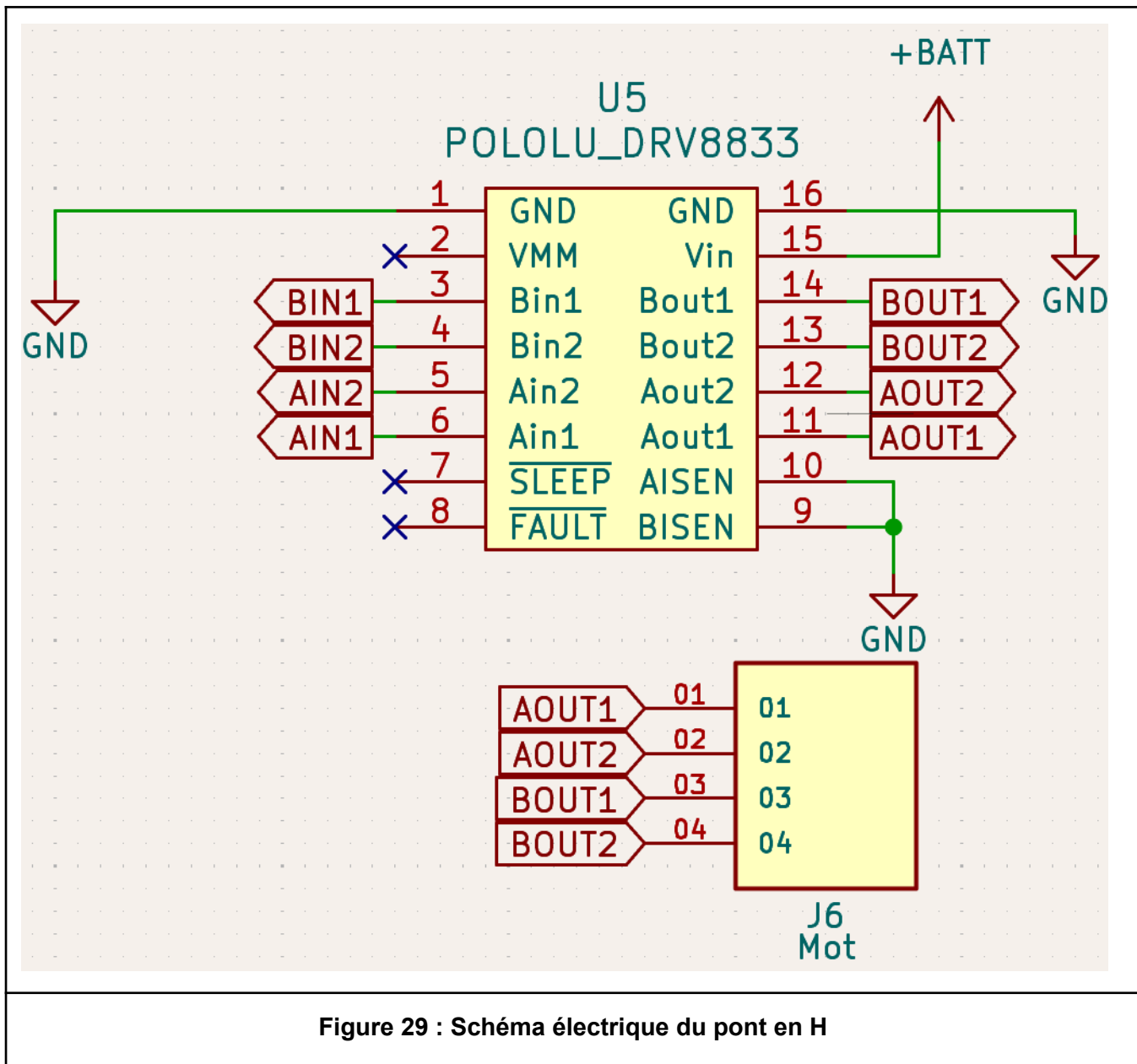


Figure 29 : Schéma électrique du pont en H

3.3.1.2 Schéma des branchements du pont en H pour les expérimentations

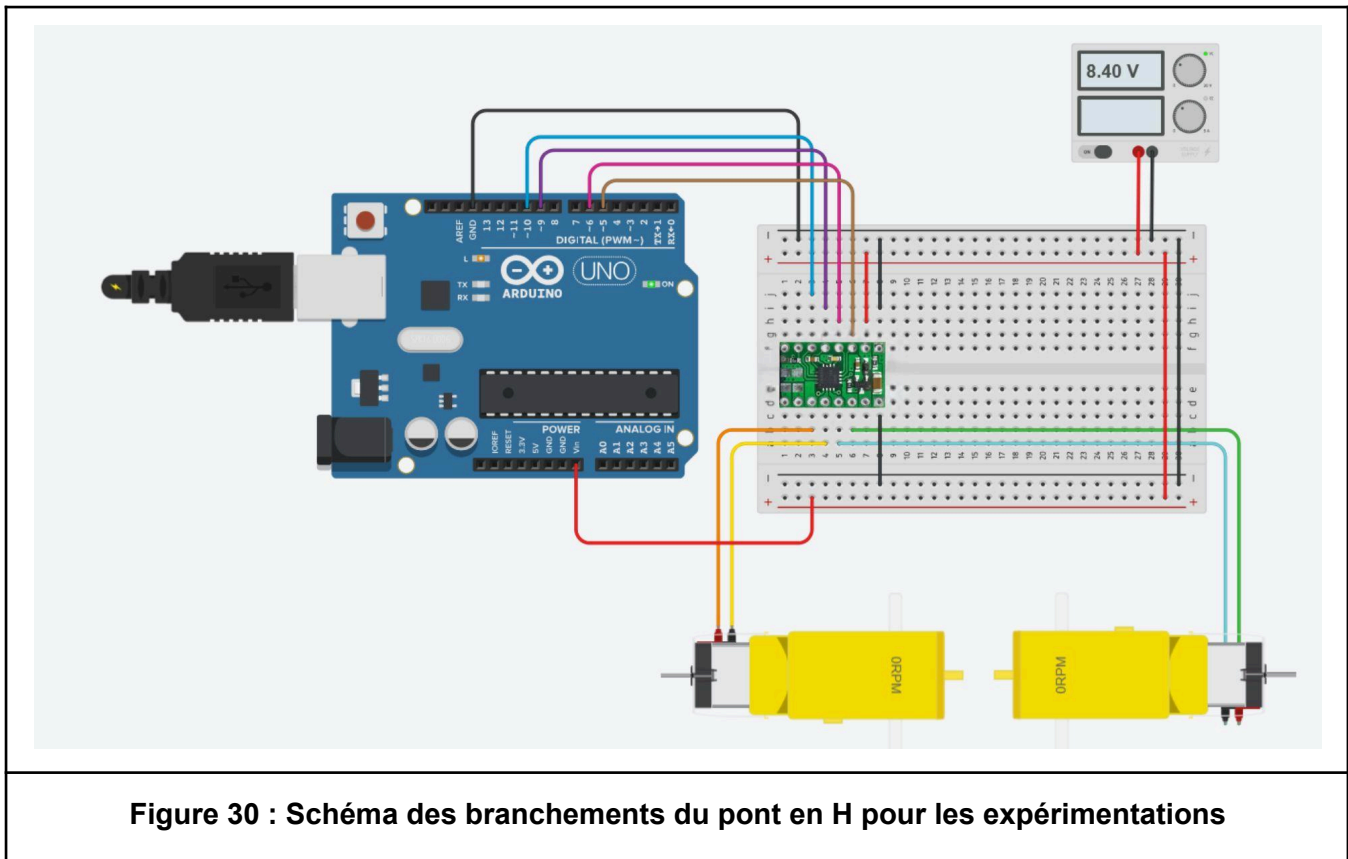


Figure 30 : Schéma des branchements du pont en H pour les expérimentations

Objectifs de l'expérimentation

Avant d'intégrer le système de motorisation dans le robot sumo final, il est essentiel de valider le bon fonctionnement du pont en H DRV8833 et de caractériser son comportement avec nos motoréducteurs. Cette phase d'expérimentation permet de tester le composant dans des conditions contrôlées, de valider le code de pilotage et d'identifier d'éventuels problèmes avant l'assemblage final.

Description du câblage expérimental

Le montage test (Figure X) comprend un Arduino UNO connecté au DRV8833 via un breadboard. Les broches de contrôle du pont en H (IN1, IN2, IN3, IN4) sont reliées aux sorties PWM de l'Arduino (broches 5, 6, 9, 10), permettant ainsi le contrôle de la vitesse et du sens de rotation des deux moteurs. L'alimentation des moteurs est assurée par une alimentation de laboratoire réglée à 8,40V, simulant la tension nominale de la batterie LiPo 2S qui équipera le robot final. Le GND de l'alimentation externe est relié au GND de l'Arduino pour établir une masse commune, indispensable au bon fonctionnement du système.

Tests réalisés

Plusieurs essais ont été menés pour valider le système :

- Test de rotation bidirectionnelle : vérification du sens de rotation de chaque moteur en inversant

les commandes des broches INx

- Test de vitesse (PWM) : validation de la variation de vitesse par modulation de largeur d'impulsion de 0 à 255
- Mesures de courant : relevé de la consommation à vide et en charge pour dimensionner correctement l'alimentation finale
- Test de charge : évaluation du couple moteur en bloquant partiellement les roues

Problématiques identifiées

La principale difficulté rencontrée concerne la consommation électrique. L'alimentation USB de l'Arduino (limitée à 500mA) s'est révélée largement insuffisante pour alimenter les moteurs, même à vide. L'utilisation d'une alimentation externe stabilisée était donc indispensable pour cette phase de test, confirmant la nécessité d'une batterie LiPo dédiée dans le montage final.

3.3.1.3 Code informatique du pont en H

```
// ----- Macro sin(x) et cos(x) -----
#define SIN_LUT(x) sinLUT[x]
#define COS_LUT(x) sinLUT[(x + 18) % 72]

// ----- Macro pour piloter un moteur -----
#define SET_MOTOR(in1, in2, pwm) ((abs(pwm) < 10) ? (analogWrite(in1, 0),
analogWrite(in2, 0)) : ((pwm) > 0 ? (analogWrite(in1, pwm), analogWrite(in2,
0)) : (analogWrite(in1, 0), analogWrite(in2, -(pwm)))))

// ----- Macro pour récupérer l'index de la LUT -----
#define GET_INDEX(angle) (((uint8_t)((angle) + 180)) / 5) % 72

// ----- Configuration des pins moteurs -----
const uint8_t IN1 = 5, IN2 = 6; // Moteur gauche
const uint8_t IN3 = 9, IN4 = 10; // Moteur droit

// ----- LUT sin 0°-355° pas 5° (centrée sur 0, amplitude ±127)
const int8_t sinLUT[72] = {
0,11,22,33,43,54,64,73,82,90,97,104,110,115,119,123,125,127,
127,127,125,123,119,115,110,104,97,90,82,73,64,54,43,33,22,11,0,-11,
-22,-33,-43,-54,-64,-73,-82,-90,-97,-104,-110,-115,-119,-123,-125,-127,-127,
-127,-125,-123,-119,-115,-110,-104,-97,-90,-82,-73,-64,-54,-43,-33,-22,-11
};

// ----- Fonction de déplacement -----
void drive(int16_t angle, int16_t pwr) {
pwr = constrain(pwr, -255, 255);
```

```

angle = constrain(angle, -180, 180);

uint8_t idx = GET_INDEX(angle);
int8_t SIN = SIN_LUT(idx);
int16_t left, right;

if (angle >= -90 && angle <= 90) {
    int16_t base_speed = pwr;
    int16_t steering = (SIN * abs(pwr)) >> 7;
    left = constrain(base_speed - steering, -255, 255);
    right = constrain(base_speed + steering, -255, 255);
} else {
    angle < 0 ? (pwr = -pwr) : pwr;
    int16_t magnitude = (abs(SIN) * pwr) >> 7;
    left = constrain(magnitude, -255, 255);
    right = constrain(-magnitude, -255, 255);
}
SET_MOTOR(IN1, IN2, left);
SET_MOTOR(IN3, IN4, right);
}

void setup() {
    Serial.begin(19200);

    // Configuration des broches en sortie
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    drive(0, 0); // Initialisation des moteurs
}

void loop() {
    /* Test des moteurs et de la fonction
    drive(0, 255);
    delay(2000);
    drive(45, 255);
    delay(2000);
    drive(-45, 255);
    delay(2000);
    drive(135, 255);

```

```

delay(2000);
drive(-135, 255);
delay(2000);
*/
}

```

Figure 31 : Code informatique du pont en H

Le code présenté constitue une implémentation optimisée de commande vectorielle pour les moteurs d'un robot sumo miniature, reposant sur une table de consultation (Look-Up Table, LUT) précalculée pour les fonctions sinus et cosinus. Cette approche, typique des systèmes embarqués à ressources limitées comme les microcontrôleurs Arduino, vise à maximiser la réactivité et la fluidité du déplacement tout en minimisant la charge de calcul en temps réel (overhead). Le robot sumo, par sa nature compétitive, exige des transitions rapides entre translation pure, rotation sur place et virages différentielles, ce qui justifie l'adoption d'une stratégie de contrôle basée sur un angle de direction et une puissance globale, plutôt que sur des vitesses individuelles pour chaque moteur.

La LUT sinLUT contient 72 valeurs entières signées sur 8 bits (de -127 à +127), correspondant aux valeurs du sinus pour des angles de 0° à 355° par pas de 5°. Chaque entrée représente donc $\sin(\theta) \times 127$, arrondi à l'entier le plus proche, ce qui permet une représentation compacte et une multiplication rapide par décalage binaire. L'utilisation d'une LUT plutôt que d'un calcul direct via $\sin()$ ou $\cos()$ s'explique par des contraintes de performance critiques : sur un microcontrôleur AVR à 16 MHz, l'appel à une fonction trigonométrique flottante peut nécessiter plusieurs centaines de cycles, alors qu'un accès mémoire à une LUT ne requiert que quelques cycles. Dans un contexte de sumo où la boucle de contrôle doit tourner à plusieurs centaines de hertz pour garantir une réactivité face à l'adversaire, cette optimisation devient déterminante.

Les macros SIN_LUT(x) et COS_LUT(x) encapsulent l'accès à la table en exploitant la périodicité et la symétrie du sinus. COS_LUT(x) est défini comme SIN_LUT((x + 18) % 72), ce qui correspond à un déphasage de 90° ($18 \times 5^\circ = 90^\circ$), évitant ainsi le stockage redondant d'une table cosinus séparée. La macro GET_INDEX(angle) normalise tout angle entré entre -180° et +180° en un indice valide dans la LUT : elle translate l'angle de +180° pour le rendre positif, divise par 5 pour obtenir l'indice, puis applique un modulo 72 pour gérer le débordement. Cette opération, réalisée uniquement avec des entiers non signés, garantit robustesse et rapidité.

La fonction centrale drive(int16_t angle, int16_t pwr) implémente une commande vectorielle différenciée selon la zone angulaire. Lorsque l'angle est compris entre -90° et +90°, le robot avance (ou recule si pwr négatif) avec une correction de direction : la vitesse de base est pwr, et un terme de braquage proportionnel à $\sin(\text{angle})$ est soustrait à un moteur et ajouté à l'autre. La multiplication ($\text{SIN} * \text{abs}(\text{pwr})$) >> 7 reconstitue efficacement le produit $\sin(\theta) \times |\text{pwr}|$ grâce à la mise à l'échelle préalable de la LUT. En dehors de cette zone ($\text{angle} \in]-180^\circ, -90^\circ] \cup [90^\circ, 180^\circ[$), le comportement bascule vers une rotation sur place : la magnitude devient proportionnelle à $|\sin(\text{angle})|$, et les moteurs tournent en sens opposé. Ce basculement garantit que le vecteur de vitesse reste aligné avec la direction demandée, même lors de demi-tours, ce qui est essentiel pour les manœuvres d'évitement ou de

contournement en combat sumo.

La macro `SET_MOTOR(in1, in2, pwm)` traduit une consigne de puissance signée en signaux PWM sur deux broches H-bridge, avec gestion du seuil mort : si $|pwm| < 10$, les deux broches sont mises à 0 pour éviter les vibrations dues aux frottements statiques ; sinon, une seule broche est activée selon le signe, l'autre restant à 0. L'usage de `analogWrite` suppose un PWM matériel 8 bits (0-255), cohérent avec la plage de `pwr` contrainte entre -255 et 255. Cette macro améliore la lisibilité et la maintenabilité du code tout en évitant les duplications.

La configuration des broches (IN1, IN2 pour le moteur gauche ; IN3, IN4 pour le droit) reflète une topologie classique de pont en H double, permettant un contrôle bidirectionnel indépendant. Le `setup()` initialise le port série à 19200 bauds (utile pour le débogage) et configure les broches en sortie, avant d'appeler `drive(0, 0)` pour garantir l'arrêt des moteurs à la mise sous tension. La boucle `loop()` est laissée commentée avec une séquence de test couvrant les directions cardinales (0° , $\pm 45^\circ$, $\pm 135^\circ$) à pleine puissance, permettant une validation fonctionnelle complète du système.

En conclusion, ce code illustre une conception sobre mais hautement optimisée pour un robot sumo autonome : l'usage systématique de macros, de LUT, d'opérations entières et de seuils morts reflète une recherche d'efficacité temporelle et énergétique, indispensable dans un environnement où chaque milliseconde et chaque mA comptent. L'approche vectorielle unifiée simplifie l'interface de haut niveau (un seul angle et une seule puissance) tout en offrant une trajectoire fluide et prévisible, critère clé pour la stratégie de combat.

3.4 Conception détaillée de la partie énergie

3.4.1 Pont diviseur de tension

Référence de conception : C1b 21, 22, 23, 24, 25, 26

Exigences client vérifiées : EXIG_AUTONOMIE, EXIG_SECUR_BATT

Rédacteurs : Théo Feugnet

Relecteurs : Arthur Blanc / Léo Jobard / Iban Irastorza / Jules Clause

3.4.1.1 Schéma électrique du pont diviseur de tension

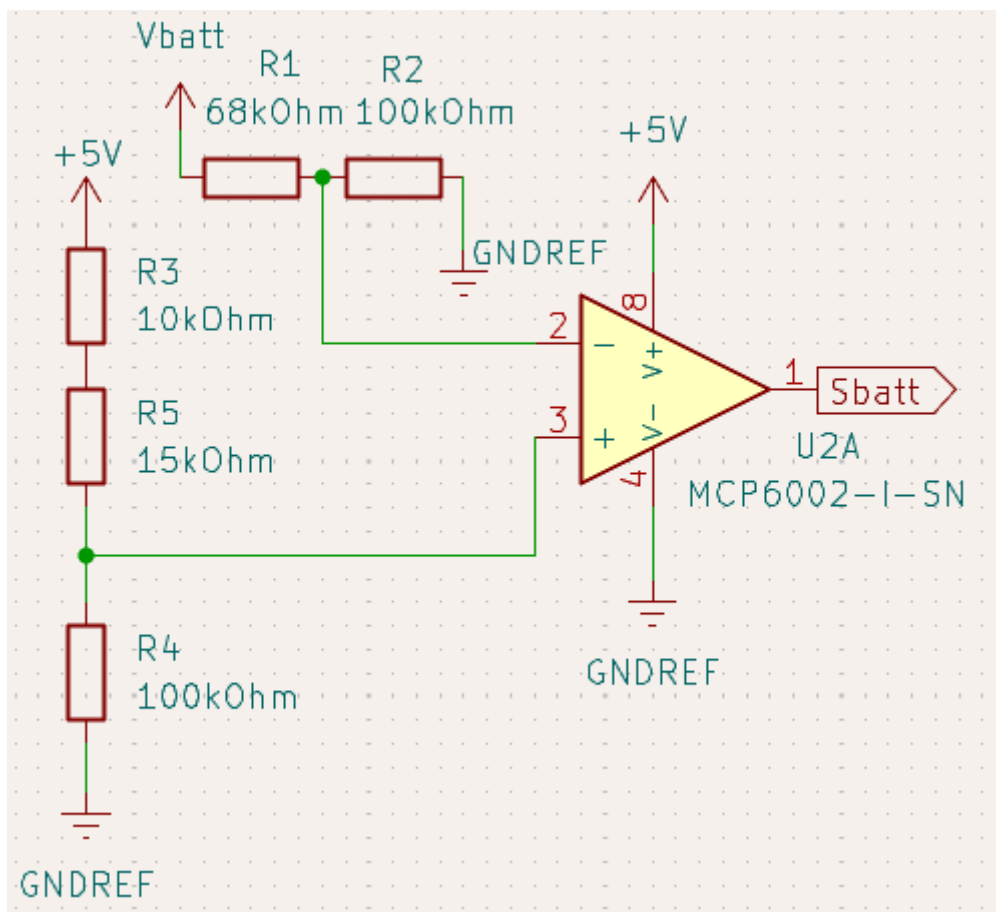


Figure 32 : Schéma électrique du seuil

Pour répondre à l'exigence EXIG_AUTONOMIE et EXIG_SECUR_BATT, nous avons décidé de mettre en place 2 ponts diviseurs de tension associés à un comparateur. L'idée est d'envoyer un signal binaire au microcontrôleur lui indiquant que la batterie est trop faible (approchant 6.7V) pour qu'il arrête les moteurs. Le choix d'aborder une valeur binaire plutôt qu'une valeur analogique, pourtant plus précise et modulable, vient du manque de pin analogiques disponibles pour notre système.

Le premier pont diviseur de tension sert de seuil indiquant au comparateur la tension limite avant

qu'il commute.

$$V_{battMax} \times V_{compSeuil} = V_{compMax} \times V_{battSeuil}$$

$$V_{compSeuil} = \frac{V_{compMax} \times V_{battSeuil}}{V_{battMax}} = \frac{5 \times 6.7}{8.4} = 3.98V$$

$$V_{compSeuil} = V_{seuil} \times \frac{R1}{R1+R2}$$

$$3.98 = 5 \times \frac{R1}{R1+R2}$$

$$\frac{R1}{R1+R2} = \frac{3.98}{5} = 0.8$$

$$\text{En posant } R1 = 100000 : \frac{100000}{100000+R2} = 0.8 \Leftrightarrow R2 = \frac{100000}{0.8} - 100000 = 25000 \text{ Ohm}$$

$R1 = 100 \text{ kOhm}$ et $R2 = 25 \text{ kOhm}$

N'ayant pas de résistances 25kOhm dans le stock, nous allons donc choisir 2 résistances en série de 10 kOhm et 15 kOhm.

Le second pont diviseur de tension sert à ramener la tension actuelle de la batterie à une valeur entre 0 et 5 V.

$$5 = V_{battMax} \times \frac{R1}{R1+R2}$$

$$5 = 8.4 \times \frac{R1}{R1+R2}$$

$$\frac{R3}{R3+R4} = \frac{5}{8.4} = 0.6$$

$$\text{En posant } R3 = 100000 : \frac{100000}{100000+R4} = 0.6 \Leftrightarrow R4 = \frac{100000}{0.6} - 100000 = 68000 \text{ Ohm}$$

En normalisant, on obtient

$R1 = 100 \text{ kOhm}$ et $R2 = 68 \text{ kOhm}$

3.4.1.2 Code informatique du pont diviseur de tension

Présenter le code des fonctions associées aux capteurs adversaires.

Expliquer en détail le fonctionnement des fonctions.

3.4.2 Régulateur de tension

Référence de conception : C1b 21, 22, 23, 24, 25, 26

Exigences client vérifiées : EXIG_AUTONOMIE, EXIG_SECUR_BATT

Rédacteurs : Théo Feugnet

Relecteurs : Arthur Blanc / Léo Jobard / Iban Irastorza / Jules Clause

3.4.2.1 Schéma électrique du régulateur de tension

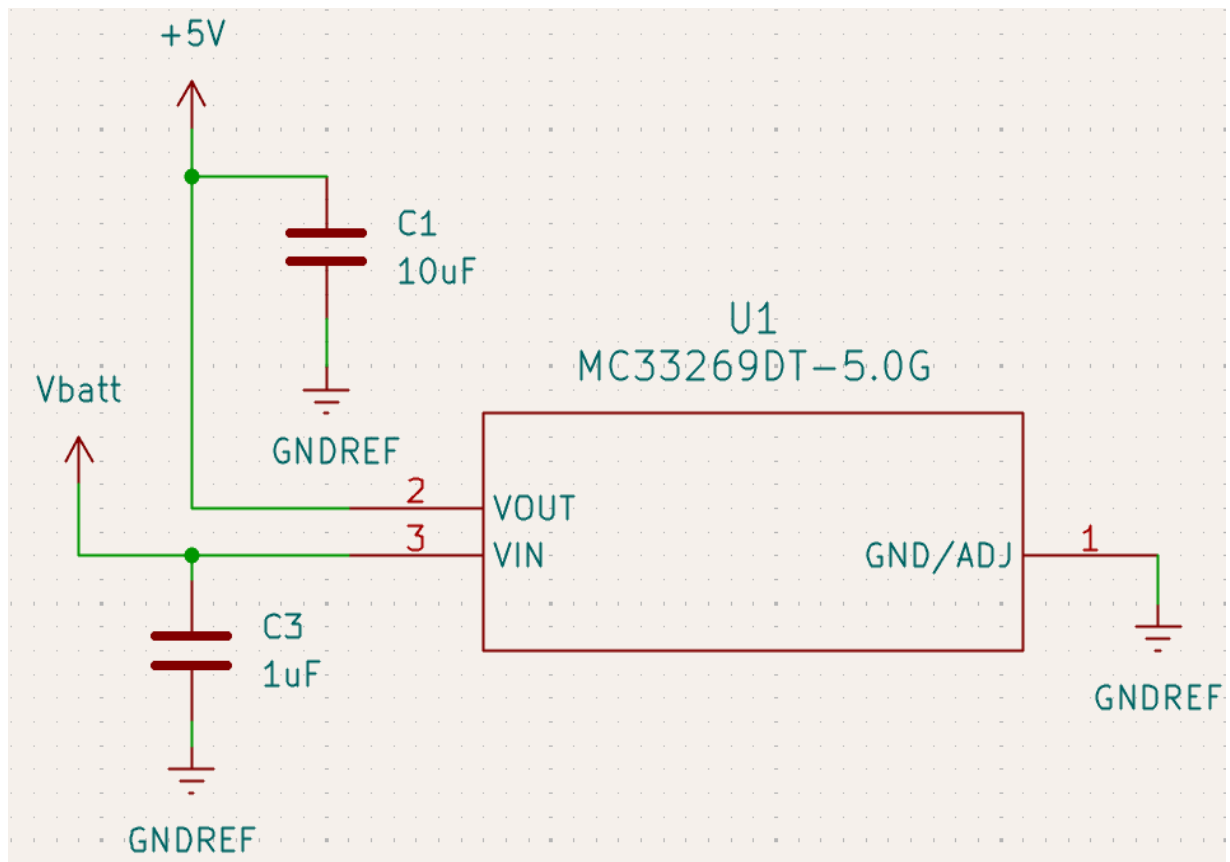


Figure 33 : Schéma électrique du régulateur linéaire

Le régulateur de tension que nous avons utilisé est le MC33269DT-5.0G. Nous avons mis en place des condensateurs de découplage afin de stabiliser la tension.

D'après la datasheet du MC33269DT-5.0G, nous avons utilisé un montage pour une tension de sortie fixe (cf. Figure 11 : Typical Fixed Output Application page 6 datasheet MC33269DT-5.0G). Nous avons mis un condensateur de découplage de 10 μF (C1) à la sortie du régulateur linéaire selon ce qui est indiqué dans la datasheet (cf. § "Applications information" page 6). Nous avons aussi choisi un condensateur de découplage (C3) de 1 μF car supérieur à 0,33 μF (cf. § "Applications information" page 6).

$$I_{\text{tot-Reg}} = 2I_{\text{capteur télémétrique}} + 2I_{\text{capteur ultrason}} + 4I_{\text{capteur contraste}} + I_{\text{capteur IR}} + I_{\text{Arduino}}$$

$$AN : I_{\text{tot}} = 2 \times 0.04 + 2 \times 0.015 + 4 \times 0.05 + 0.001 + 0.045 = 0.356 \text{ A} = 356 \text{ mA}$$

Le régulateur linéaire MC33269DT-5.0G peut supporter un courant max de 800 mA (cf. § "ELECTRICAL CHARACTERISTICS" page 2) : il est suffisamment faible pour que cela n'abime pas le régulateur linéaire.

3.4.3 Protection contre les courants inverse avec une diode anti-retour

Référence de conception : C1b 21, 22, 23, 24, 25, 26

Exigences client vérifiées : EXIG_AUTONOMIE, EXIG_SECUR_BATT

Rédacteurs : Théo Feugnet

Relecteurs : Arthur Blanc / Léo Jobard / Iban Irastorza / Jules Clause

3.4.2.1 Schéma électrique du régulateur de tension

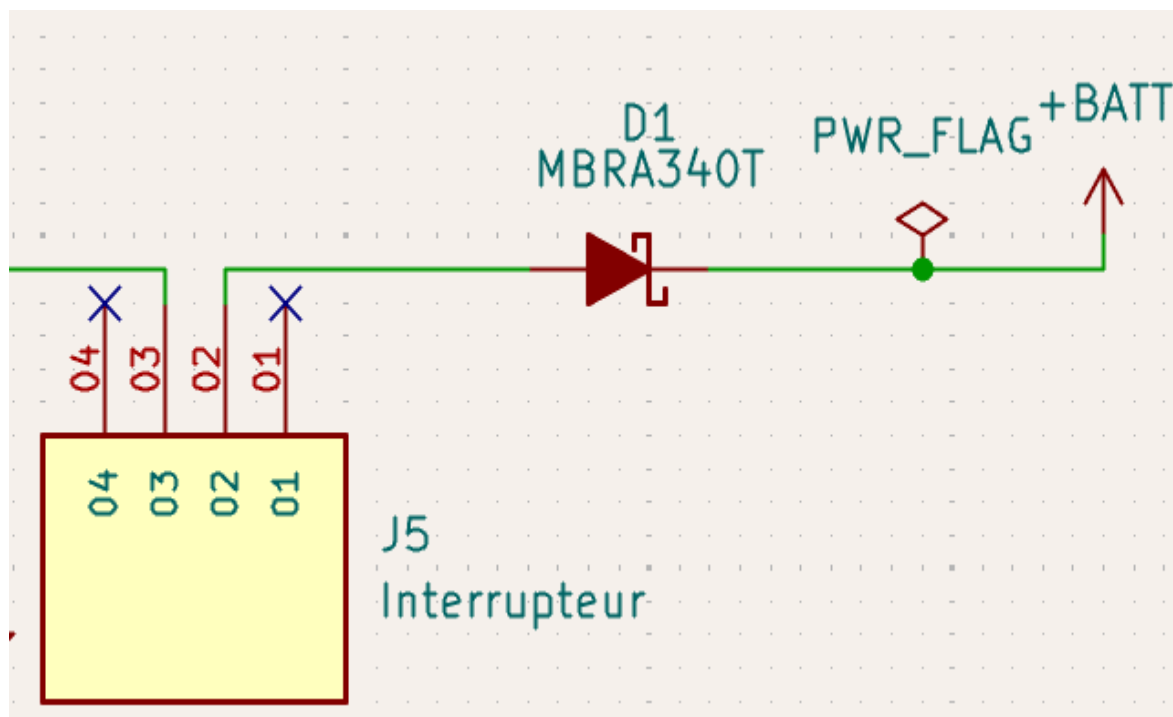


Figure 34 : Schéma électrique de la diode anti-retour

Afin de protéger la carte d'un mauvais branchement de la batterie, nous avons décidé d'ajouter une diode de protection anti-retour en sortie de l'interrupteur. Dû aux forts courants traversés par la diode (plus de 631mA), nous nous sommes orientés vers une diode Schottky, plus résistante mais aussi plus chère.

Notre choix est la diode Schottky MBRA340T3G car elle est capable de supporter :

- 40 V en fonctionnement inverse, valeur bien supérieure à la tension maximum que la batterie peut délivrer (8.4V)
- 3 A en fonctionnement normal et 100 A en cas de pic de courant, valeur bien supérieure au courant maximum (631 mA) consommé par notre robot

3.5 Conception détaillée de la partie traitement

3.5.1 Le microcontrôleur ATMEGA328P-PU

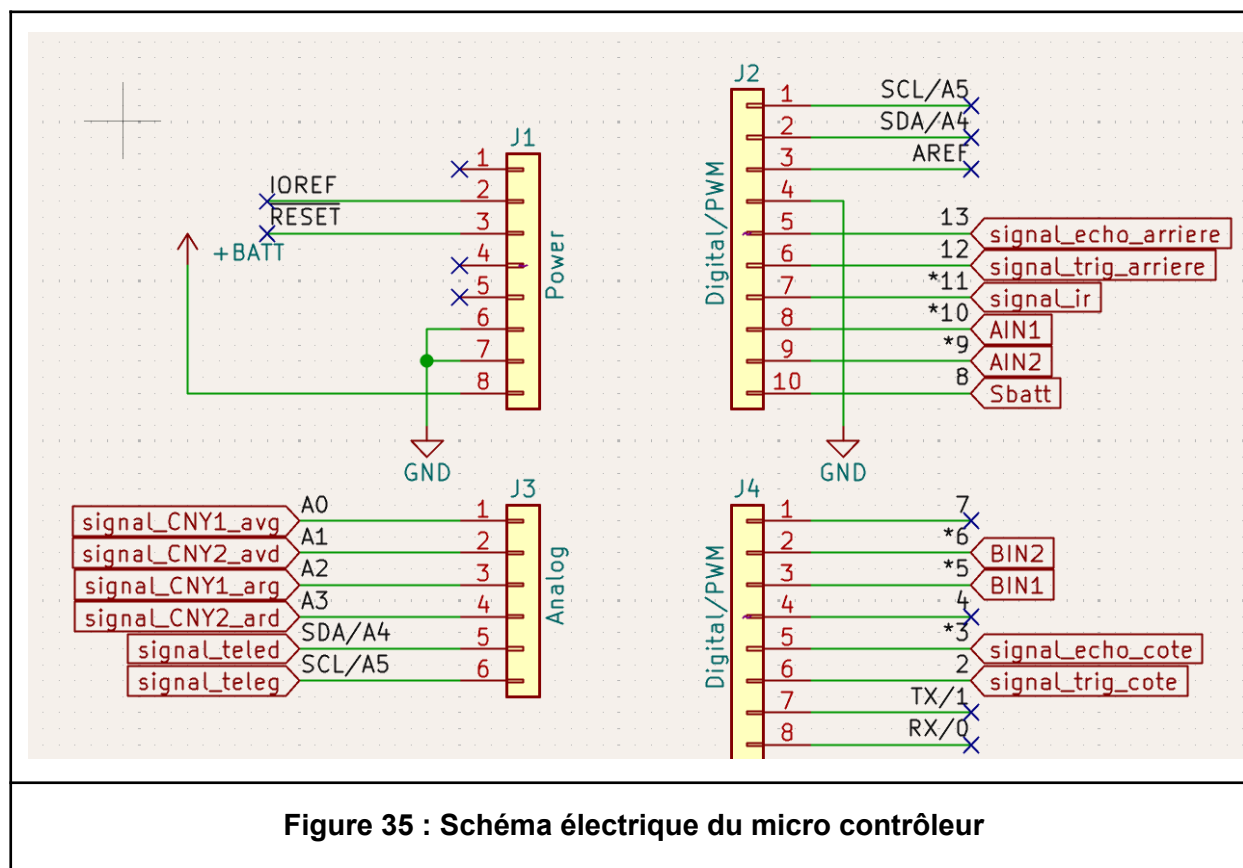
Référence de conception : C1b 21, 22, 23, 24, 25, 26

Exigences client vérifiées : EXIG_COMPORTEMENT

Rédacteurs : Théo Feugnet

Relecteurs : Arthur Blanc / Léo Jobard / Iban Irastorza / Jules Clause

3.5.1.1 Schéma électrique du microcontrôleur



En accord avec la conception préliminaire, nous avons décidé d'utiliser un Arduino uno. L'arduino uno possède déjà tout le nécessaire pour faire fonctionner correctement l'ATMEGA328P, nous n'aurons donc pas à gérer l'implémentation d'un quartz, de condensateurs de découplage et du régulateur linéaire.

Nous nous contenterons d'alimenter l'Arduino uno et de relier les pins aux connecteurs du shield.

L'Arduino Uno possède déjà son propre régulateur linéaire ce qui indique que nous n'avons pas besoin de lui fournir une tension stable. La tension d'alimentation nécessaire au bon fonctionnement de l'Arduino Uno est de 7 V à 12 V si alimentation par le pin Vin de l'Arduino Uno. Vu que notre batterie

délivre une tension entre 6.7 V et 8.4 V, nous pouvons l'alimenter malgré une légère sous-tension sur la fin de la batterie.

Le choix des pins s'est fait sur deux facteurs : le type de pin nécessaire (PWM, Analog ou GPIO) et sur leur facilité de routage. Ainsi les pins de signal des capteurs et du pont en H sont souvent placés au plus proche l'un des autres afin de faciliter leur routage.

3.5.1.2 Code informatique du microcontrôleur

Aucun code pour le moment

4. Dériskage des solutions techniques retenues

Ce chapitre détaille les activités de dériskage des solutions techniques retenues : simulation et/ou prototypage rapide. Il constitue une preuve partielle de la conformité du produit. Chaque paragraphe de l'étude fait donc clairement référence aux exigences client issues du [CDC].

Il permet également de confirmer les résultats théoriques effectués aux paragraphes 2 et 3 en vérifiant le fonctionnement à travers des simulations et/ou prototypages rapides. Pour chaque simulation et/ou prototypage rapide est renseigné le protocole de mise en œuvre. Les résultats des simulations et/ou prototypages rapides sont confrontés aux résultats de l'étude théorique.

L'ensemble des fichiers est disponible dans le dossier : renseignez ici le chemin du dossier où sont situés les fichiers de simulation et/ou prototypage rapide du projet.

4.1 Dériskage des mouvements du robot

Référence de la simulation : SIM_ACT_1

Exigences client vérifiées : EXIG_DEPLACEMENT, EXIG_COMPORTEMENT

But de l'essai :

Valider le bon fonctionnement du pont en H (Pololu DRV8833) avec les deux motoréducteurs GM9 + RM3 imposés : vérifier la rotation bidirectionnelle des deux moteurs, la variation de vitesse par PWM, la fluidité des mouvements vectoriels (avance, recul, virages, rotation sur place) et confirmer que le courant consommé reste dans les limites du composant et de l'alimentation.

Fichiers :

- Fichier contenant le code de test : [drive.ino](#)
- Photo du montage de prototypage :  action_tinkercad.png x

Procédure de simulation / prototypage rapide :

Montage réalisé sur breadboard + Arduino Uno R3 :

- Pont en H DRV8833 alimenté en VMOT = 8,40 V (alimentation de laboratoire simulant la LiPo 2S chargée)
- Broches de contrôle : IN1 → D5, IN2 → D6, IN3 → D9, IN4 → D10 (broches PWM)
- Moteurs gauche et droit branchés sur OUT1/OUT2 et OUT3/OUT4

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ22 Révision : 1 – 07/10/2025	46/55
----------------------------------	---	-------

Robot Mini-Sumo (RMS)

- Masse commune entre Arduino, DRV8833 et alimentation externe
- Code drive.ino chargé (celui décrit dans le §3.3.1.3 du DDC avec la LUT sinus et la fonction drive(angle, pwr))

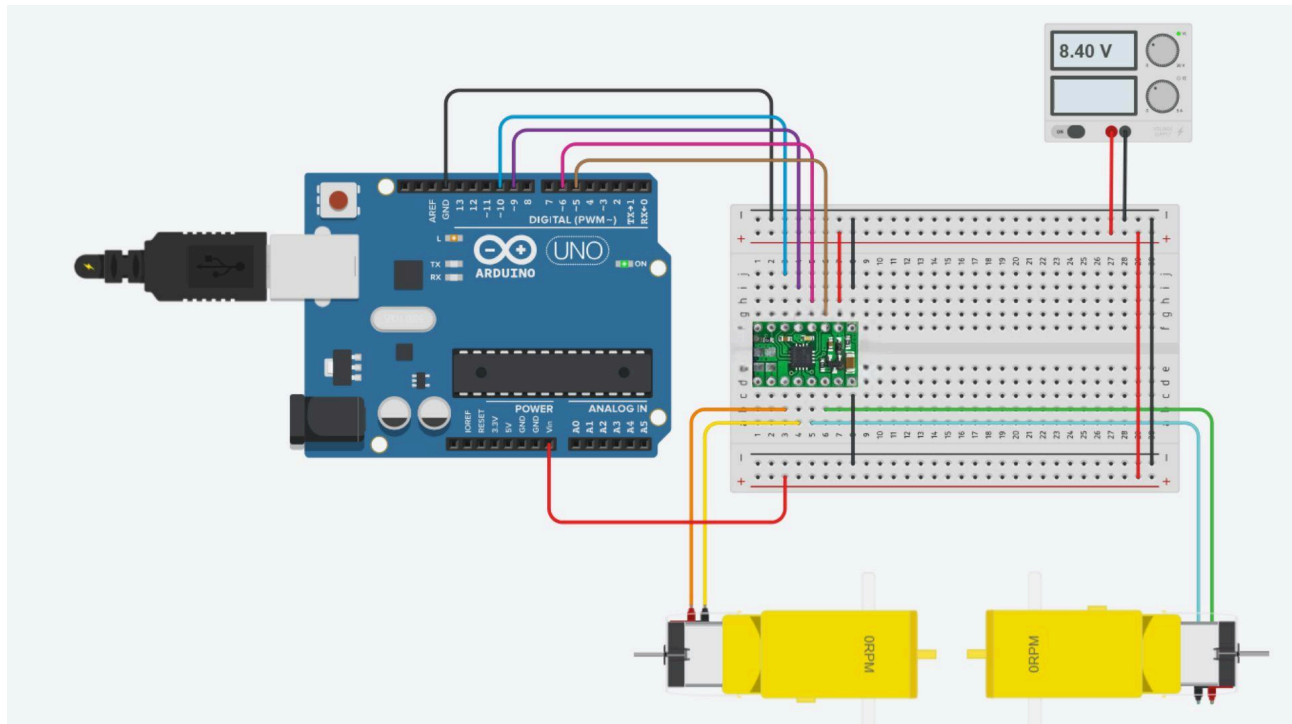


Figure 36 : Schéma des branchements du pont en H pour les expérimentations

Tests réalisés en boucle :

1. Translation avant/arrière (angle = 0° et 180° , pwr = 255)
2. Rotation sur place horaire et anti-horaire (angle = 90° et -90° , pwr = 255)
3. Virages progressifs (angle de -85° à $+85^\circ$ par pas de 5°)
4. Mesure du courant moteur à vide et en charge (roues bloquées) avec multimètre en série

Résultats attendus :

Grandeur	Valeur attendue	Tolérance
Rotation bidirectionnelle	Les 2 moteurs tournent dans les 2 sens	-
Variation de vitesse PWM	Vitesse perceptible de 0 à 100 %	-

Résultats obtenus :

Grandeur	Valeur attendue	Conformité
Rotation bidirectionnelle	OK sur les 2 moteurs	Conforme
Variation de vitesse PWM	Variation fluide et linéaire de 0 à 100 %	Conforme

Statut de l'essai : Conforme

Problèmes rencontrés :

1. Au tout début, les moteurs ne tournaient pas : le GND de l'alimentation externe n'était pas relié au GND de l'Arduino → tension flottante sur les INx du DRV8833. Problème résolu en reliant correctement les masses.
2. À pleine puissance ($pwr = 255$) pendant plus de 8–10 secondes en charge, le DRV8833 chauffe fortement ($\approx 65\text{ }^{\circ}\text{C}$). Ce n'est pas bloquant pour un match de $3 \times 30\text{ s}$ maximum, mais nous avons tout de même décidé d'ajouter (sur la version finale du shield) un petit dissipateur adhésif sur le DRV8833 par précaution.
3. Très léger « buzz » audible aux faibles PWM ($< 40/255$) dû au frottement statique des motoréducteurs. Le seuil mort à 10 dans le code (macro SET_MOTOR) élimine complètement ce bruit sans perte notable de réactivité.

Conclusion partielle

Le DRV8833 associé au code vectoriel avec LUT sinus répond parfaitement aux exigences EXIG_DEPLACEMENT et EXIG_COMPORTEMENT. Les courants restent très en deçà des limites du composant (2 A par canal) et de la batterie (800 mAh). Le système de commande par angle + puissance est validé et sera conservé tel quel pour le robot final.

4.2 Dérivage des capteurs du robot

Référence de la simulation : SIM 1

Exigences client vérifiées : Renseignez ici les références des exigences client auxquelles le paragraphe de simulation / prototypage rapide ci-dessous fait référence.

But de l'essai : Décrivez brièvement le but de la simulation / prototypage rapide vis-à-vis du cahier des charges.

Fichiers : Renseignez ici le nom des fichiers associés à la simulation / prototypage rapide de manière à pouvoir rejouer la simulation / prototypage rapide en cas de besoin au cours et après le développement du produit.

Procédure de simulation :

Pour nous assurer du bon fonctionnement du capteur, nous avons effectué des tests de mesures par prototypage rapide. Nous avons fait le montage suivant :

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ22 Révision : 1 – 07/10/2025	48/55
----------------------------------	---	-------

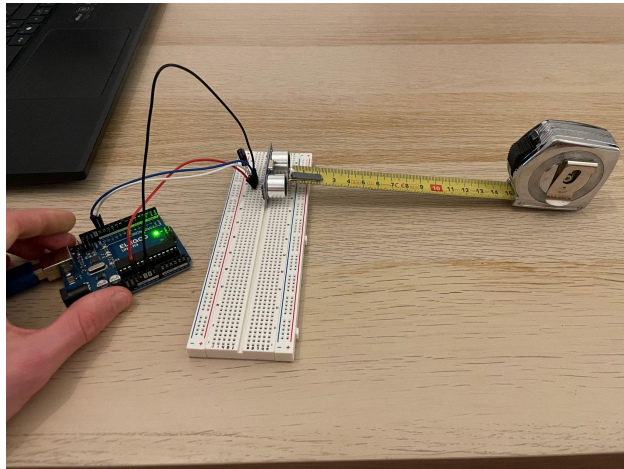


Figure 37 : prototypage rapide du HCSR04

Nous avons calculé la moyenne des erreurs relatives des valeurs renvoyées sur le moniteur série (converties en cm) par rapport aux vraies mesures effectuées au mètre :

Distance réelle	5 cm	15 cm	40 cm	70 cm
Distance affichée	4 cm	16 cm	41 cm	69 cm
erreur relative	-20%	6,6%	2,5%	-1,4%

Nous avons donc une erreur relative globale de -4,3%, qui est satisfaisante pour détecter une présence d'adversaire d'autant plus que les valeurs n'ont pas besoin d'une grande précision mais plutôt d'une certaine stabilité afin d'être triées par un seuil de 70 cm.

Statut de l'essai : conforme

Problèmes rencontrés :

Mentionnez ici les problèmes que vous avez rencontrés. Précisez les solutions que vous avez adoptées pour résoudre les problèmes, ou éventuellement les problèmes qui persistent encore lors de la rédaction de ce document. Ce paragraphe n'est pas à négliger, le client y apporte toujours beaucoup d'importance. Il permet de tracer les éventuelles modifications apportées au produit pendant son développement. On peut également informer le client des limitations du produit qui lui est livré. Il sera ainsi conscient des problèmes encore non résolus et ne perdra pas un temps précieux à comprendre pourquoi cela ne fonctionne pas. Ceci est une démarche de transparence entre un fournisseur et un client qui contribuera à la qualité finale du produit et à la satisfaction du client.

Référence de la simulation : SIM 2

Exigences client vérifiées : EXIG_CARTE

But de l'essai : Vérifier que les valeurs correspondent bien à la réalité.

Procédure de simulation :

Pour nous assurer du bon fonctionnement du capteur, nous avons effectué des tests de mesures par prototypage rapide. Nous avons fait le montage suivant :

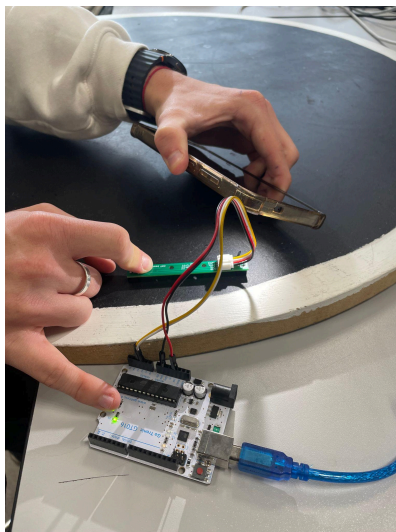


Figure 38 : prototypage rapide du CNY70

Nous avons ensuite vérifié la fiabilité de notre code et de notre capteur avec un code test **A FINIR**:

Couleur réelle	Blanc	Noir
Couleur mesurée	blanc	noir

Robot Mini-Sumo (RMS)

COM11

00000000003

```

Gauche avant : Blanc      Droite avant : Noir
Gauche arriere : Blanc   Droite arriere : Noir
-----
Gauche avant : Blanc      Droite avant : Noir
Gauche arriere : Blanc   Droite arriere : Noir
-----
Gauche avant : Blanc      Droite avant : Noir
Gauche arriere : Blanc   Droite arriere : Noir
-----
Gauche avant : Blanc      Droite avant : Noir
Gauche arriere : Blanc   Droite arriere : Noir
-----
Gauche avant : Blanc      Droite avant : Noir
Gauche arriere : Blanc   Droite arriere : Noir
-----
Gauche avant : Blanc      Droite avant : Noir
Gauche arriere : Blanc   Droite arriere : Noir
-----

```

☐ Défilement automatique ☐ Afficher l'horodatage

Figure 39 : moniteur série du test CNY70

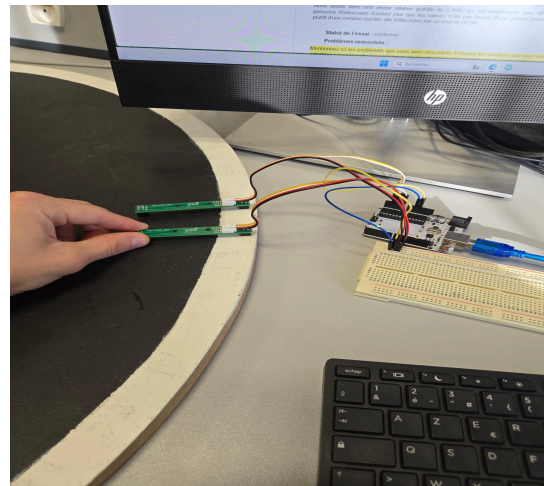


Figure 40 : Câblage du prototype

On constate que le code est bien correct avec un seuil à 800.

Statut de l'essai : conforme

Problèmes rencontrés :

Aucun problème rencontré

Référence de la simulation : SIM 2

Exigences client vérifiées : Renseignez ici les références des exigences client auxquelles le paragraphe de simulation / prototypage rapide ci-dessous fait référence.

But de l'essai : Décrivez brièvement le but de la simulation / prototypage rapide vis-à-vis du cahier des charges.

Fichiers : Renseignez ici le nom des fichiers associés à la simulation / prototypage rapide de manière à pouvoir rejouer la simulation / prototypage rapide en cas de besoin au cours et après le développement du produit.

Procédure de simulation :

Pour nous assurer du bon fonctionnement du capteur, nous avons effectué des tests de mesures par prototypage rapide. Nous avons fait le montage ci-bas. Nous avons créé un code test qui permet de lire les valeurs de distance sur le moniteur série :

```
void loop() {
  float distance_moy = 0;
  for (int i = 0; i < 3; i++) {
    int valeurBrute = analogRead(pinCapteurGauche);
```

Robot Mini-Sumo (RMS)

```
float tension = (valeurBrute / 1023.0) * referenceVoltage;
float distance = 12 * pow(tension, -1);
distance_moy += distance;
}
distance_moy = round(distance_moy / 3);
Serial.print(distance_moy);
Serial.println("");
delay(500);}
```

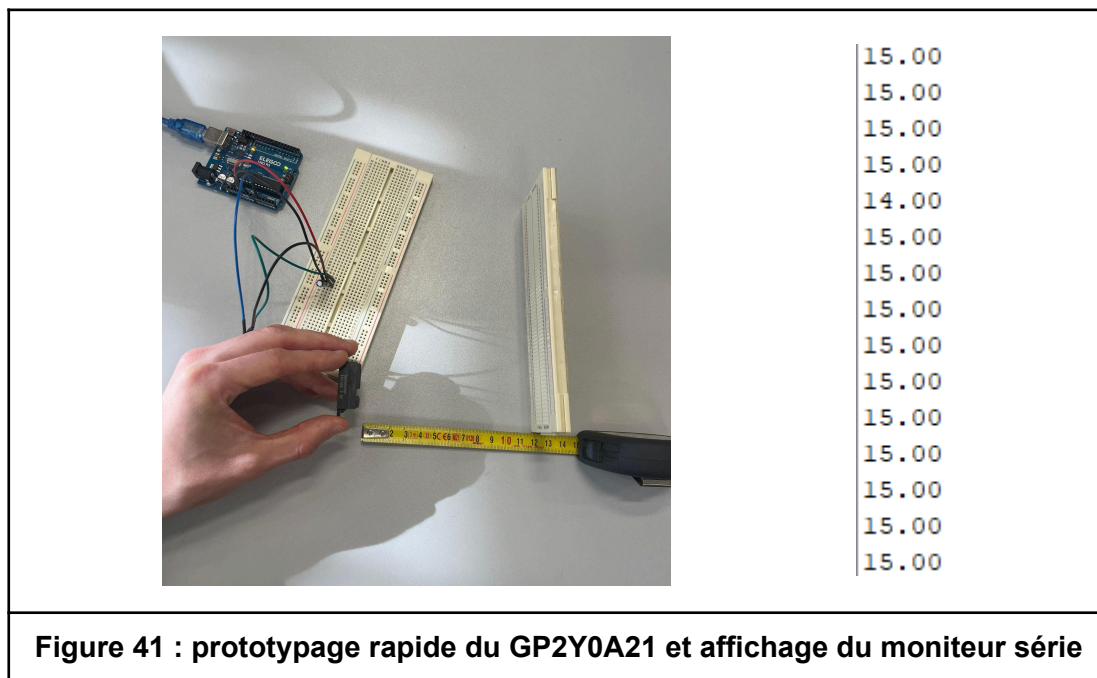


Figure 41 : prototypage rapide du GP2Y0A21 et affichage du moniteur série

Nous avons relevé les valeurs suivantes :

Distance réelle	10 cm	15 cm	25 cm	40 cm	60 cm
Distance affichée	10 cm	15 cm	26 cm	38 cm	63 cm
erreur relative	0 %	0%	4%	-5%	5%

L'erreur relative globale est de 1% et est convenable d'autant plus que les valeurs n'ont pas besoin d'une grande précision mais plutôt d'une certaine stabilité afin d'être triées par un seuil de 50 cm.

Statut de l'essai : conforme

Problèmes rencontrés :

Robot Mini-Sumo (RMS)

Lorsque nous utilisons la formule de valeur normalisée : $22.239 * \text{pow}(\text{tension}, -1)$. Nous avons pu mesurer les valeurs suivantes :

Distance réelle	10 cm	25 cm	40 cm	60 cm
Distance affichée	19 cm	46 cm	51 cm	99 cm

Nous avons remarqué que la précision était faible. C'est pour cela que nous avons expérimentalement ajusté le facteur 22,239 à 12 pour obtenir des résultats cohérents sur notre plage de mesures.

capteur IR

```
11:46:14.341 -> 0
11:46:14.563 -> 1
11:46:14.563 -> 0
11:46:14.748 -> 0
11:46:14.937 -> 1
11:46:14.937 -> 0
11:46:15.160 -> 1
11:46:15.160 -> 0
11:46:15.346 -> 0
```

4.3 Conclusion de la simulation / prototypage rapide du produit

Concluez sur la simulation / prototypage rapide du produit en insistant sur les non-conformités et les solutions possibles. Le texte ci-dessous est un exemple.

Les simulations / prototypages rapides effectués dans cette partie ont permis de valider les choix et dimensionnement de la phase de conception.

5. Conclusion de la conception du produit

Concluez sur la conception du produit vis-à-vis des exigences client en insistant plus particulièrement sur les non-conformités identifiées, les causes possibles et les solutions envisagées.

6. Matrice de conformité du produit

Ce chapitre synthétise par l'intermédiaire d'un tableau la conformité du produit développé par rapport aux exigences issues du Cahier des Charges.

Exigence	Méthodes Vérification	Eléments vérifiant l'exigence	Statut
EXIG_ENERGIE	Conception Dérivage Conception/Fab. Vérification	PRC01 CCPT05 FAB02	Conf.